

Embedded Software Engineering

Simon Künzli, Tobias Welti
HS 2025

Agenda HS25

Vorlesung (welo)

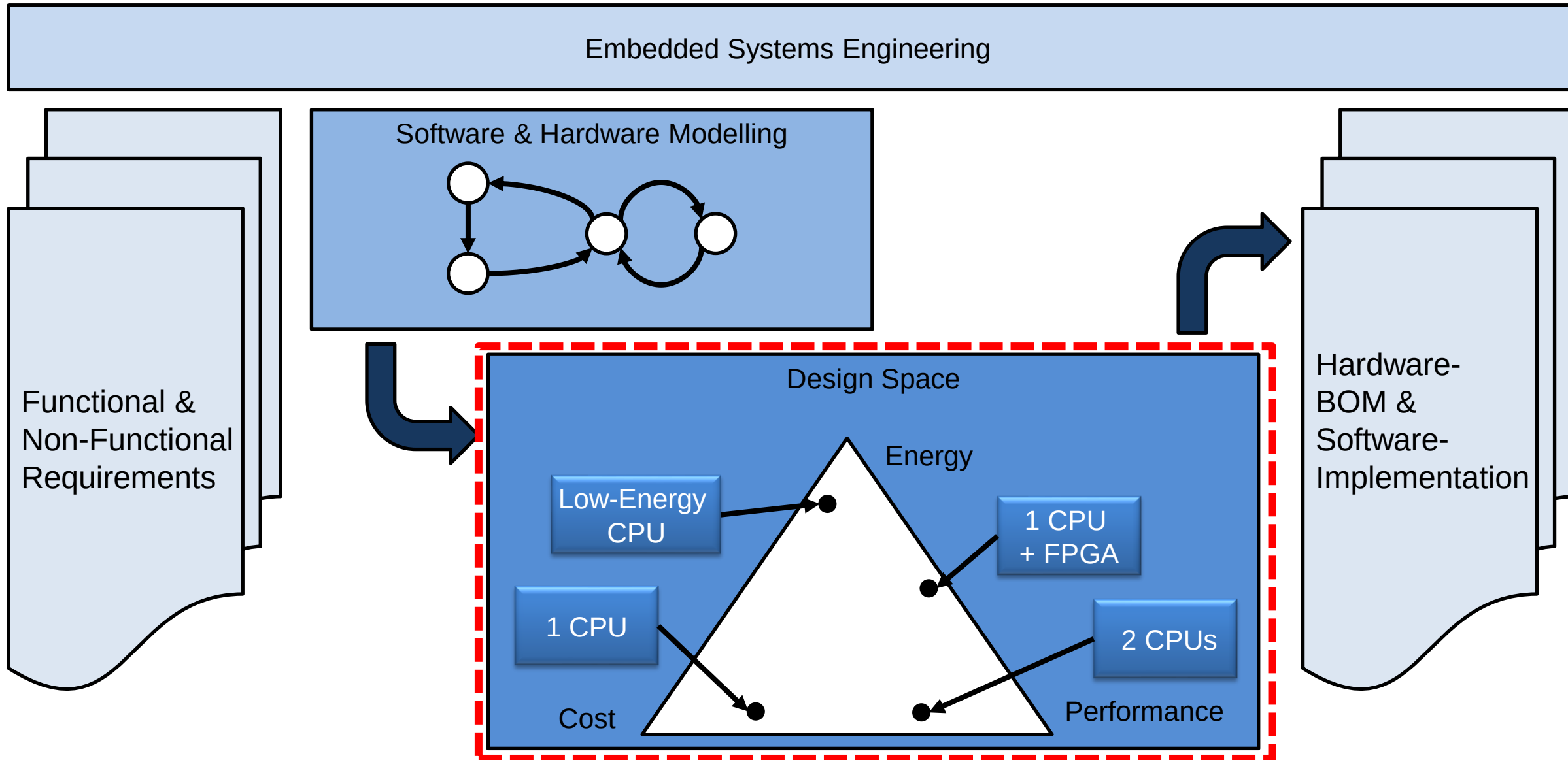
Vorlesung (kuex)

Übungen / Praktika

Stand:
27.08.25

wk #	Do, 10:00 - 11:35	Do, 08:00 – 09:35 / Do 12:00 – 13:35	
01 15.09.	Modulorganisation & Einführung Embedded Systems	No Lab	
02 22.09.	SW-Paradigmen für Embedded Systems	No Lab	P1: First steps U96
03 29.09.	Requirements für Embedded Systems	P1: First steps U96	P1: Finish Lab
04 06.10.	Software-Entwicklung für ES (Modellierung, SW-Specs)	P1: Finish Lab	U1: Requirements
05 13.10.	Entwicklungsprozesse / CI/CD	U1: Requirements	U2: Modeling
06 20.10.	Non-functional Requirements	U2: Modeling	U3: Implementation & Test
07 27.10.	Visite Fertigung Siemens	Visite Fertigung Siemens	
08 03.11.	Fokus "Energie« / andere Prozessoren?	U3: Implementation & Test	P2: Energy Consumption
09 10.11.	Fokus "Performance"	P2: Energy Consumption	P3: AES in SW, incl. Measurement on R5
10 17.11.	Einführung FPGA	P3: AES in SW, incl. Measurement on R5	P3: Finish Lab
11 24.11.	Einführung Design Space Exploration (DSE)	P3: Finish Lab	P4: AES in FPGA and HW-Accelerator
12 01.12.	DSE: Kostenfunktionen und Suche im Entwurfsraum	P4: AES in FPGA and HW-Accelerator	P4: Finish Lab
13 08.12.	Einführung RTOS	P4: Finish Lab	P5: DSE
14 15.12.	Dual-Prozessor-Systeme mit Uniform Memory Access	P5: DSE	No Lab

Advance Organizer ESE

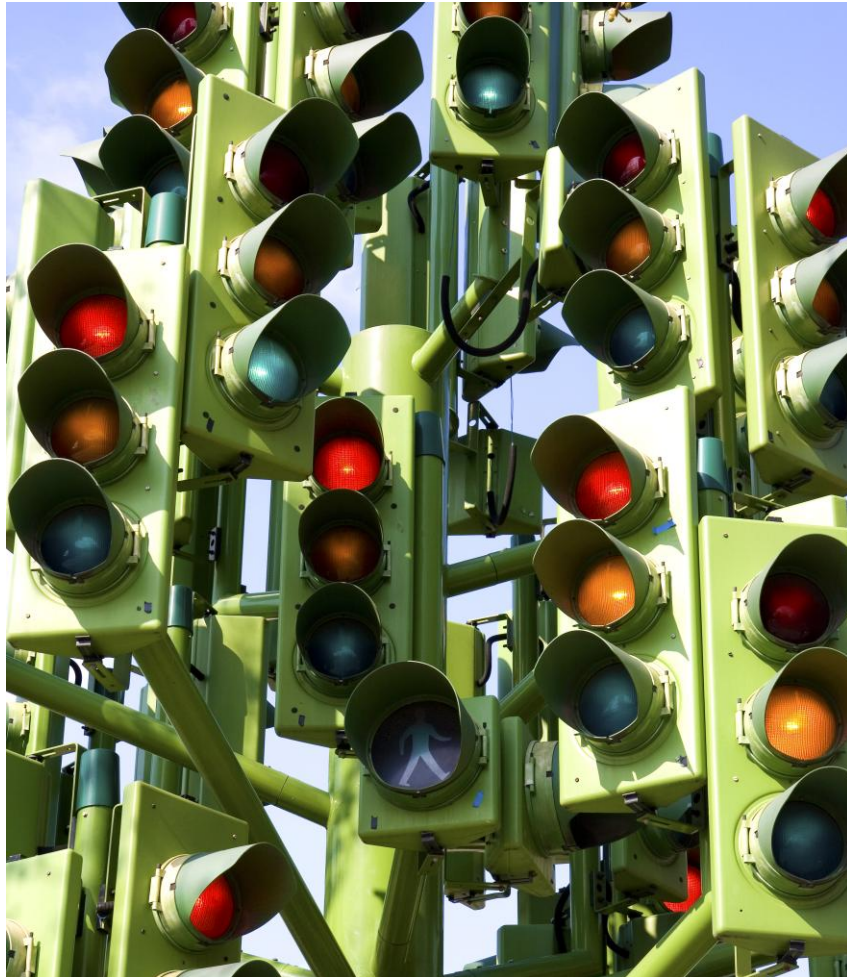


Learning Objectives

After today's lesson, you ...

- ... will be able to explain what design space exploration is about
- ... know methods of how different objectives for ES can be evaluated
- ... can explain what an architecture template, allocation, decision space, and objective space is
- ... know how to compare various Embedded System solutions
- ... will be able to perform a design space exploration

Example 1: Traffic Light Control System



Source: Colourbox.de



Source: Colourbox.de

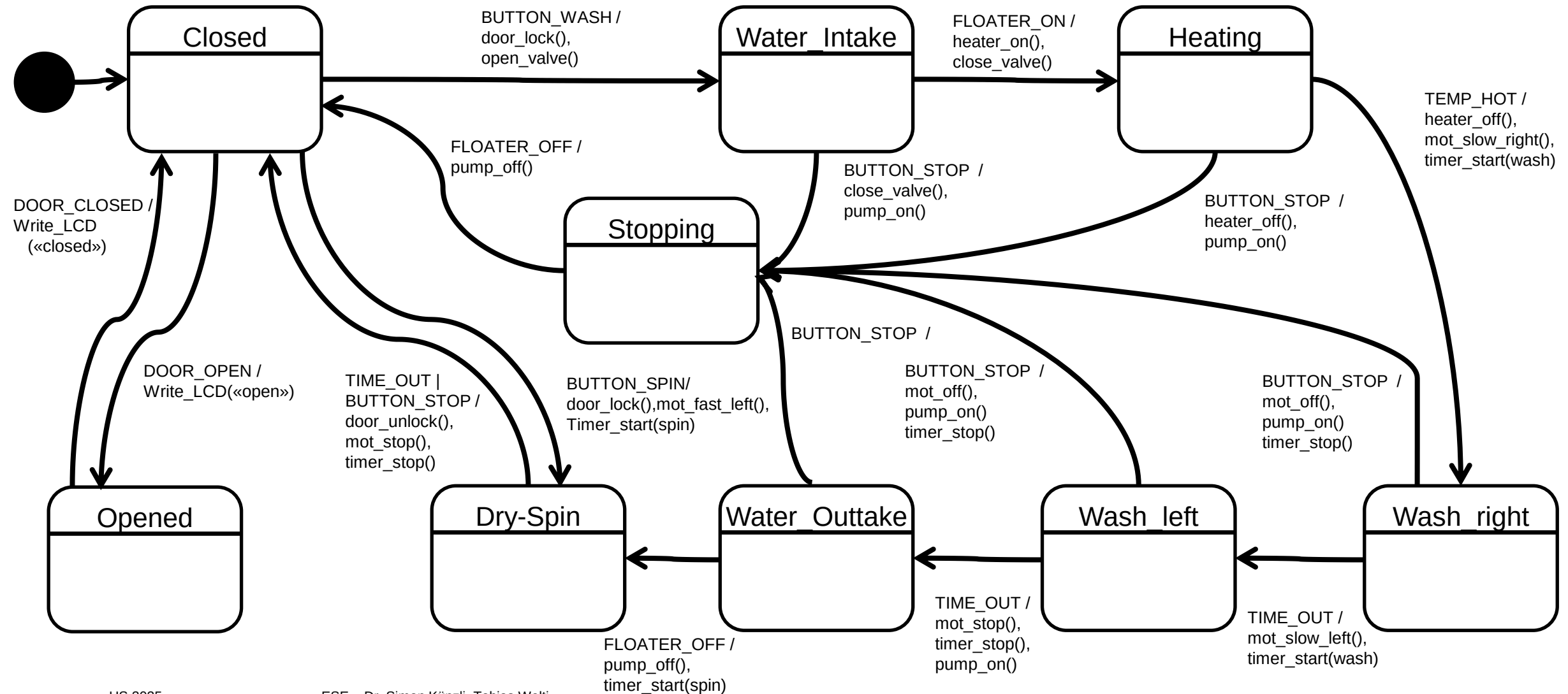
Example 2: Washing Machine

- **Control System for Washing Machine**
- **This will be the topic of the next to exercises (U1, U2)**
- **For these first 2 exercises, we will work with well-known platform: the CT1 boards**
- **Check whether you have Keil uVision still installed on your computer. There are a few computers in TE 507 with uVision installed**



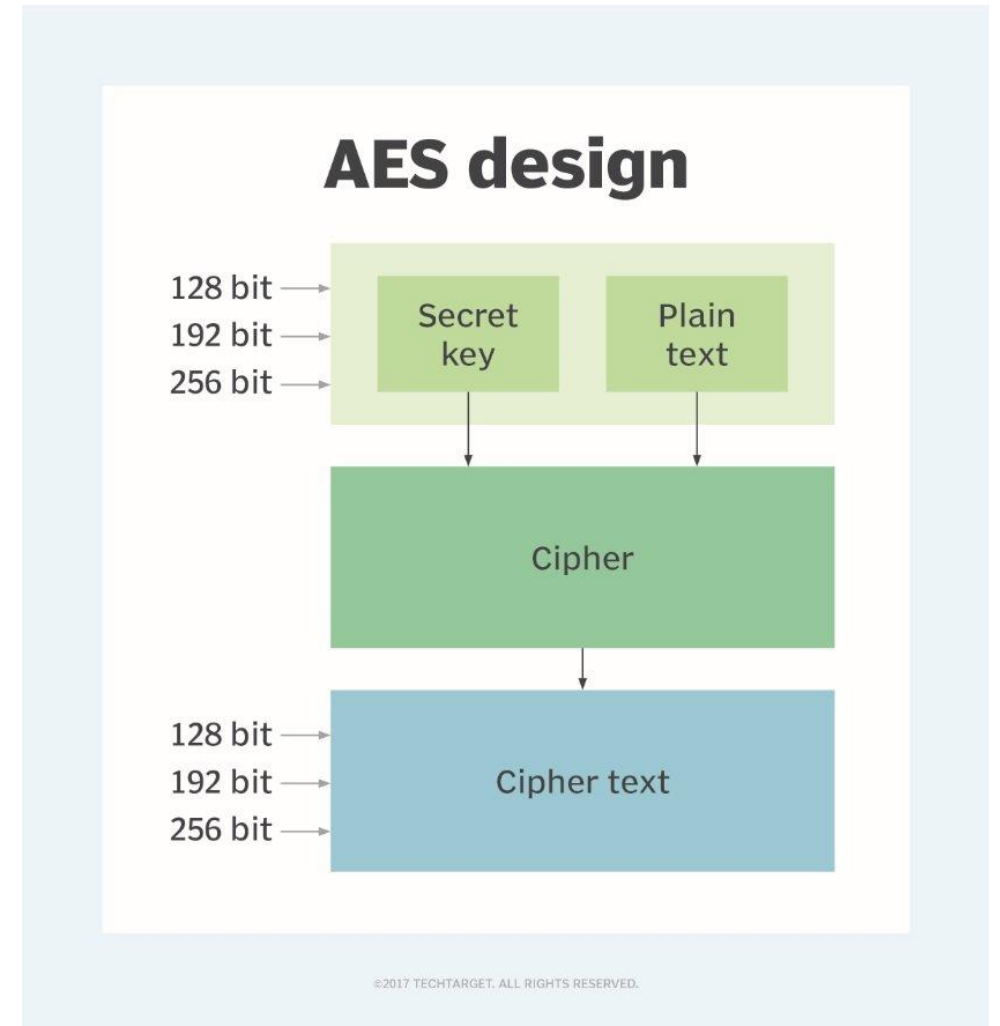
Source: Colourbox.de

Washing Machine – Possible UML State Machine



Example 3: AES Encryption/Decryption

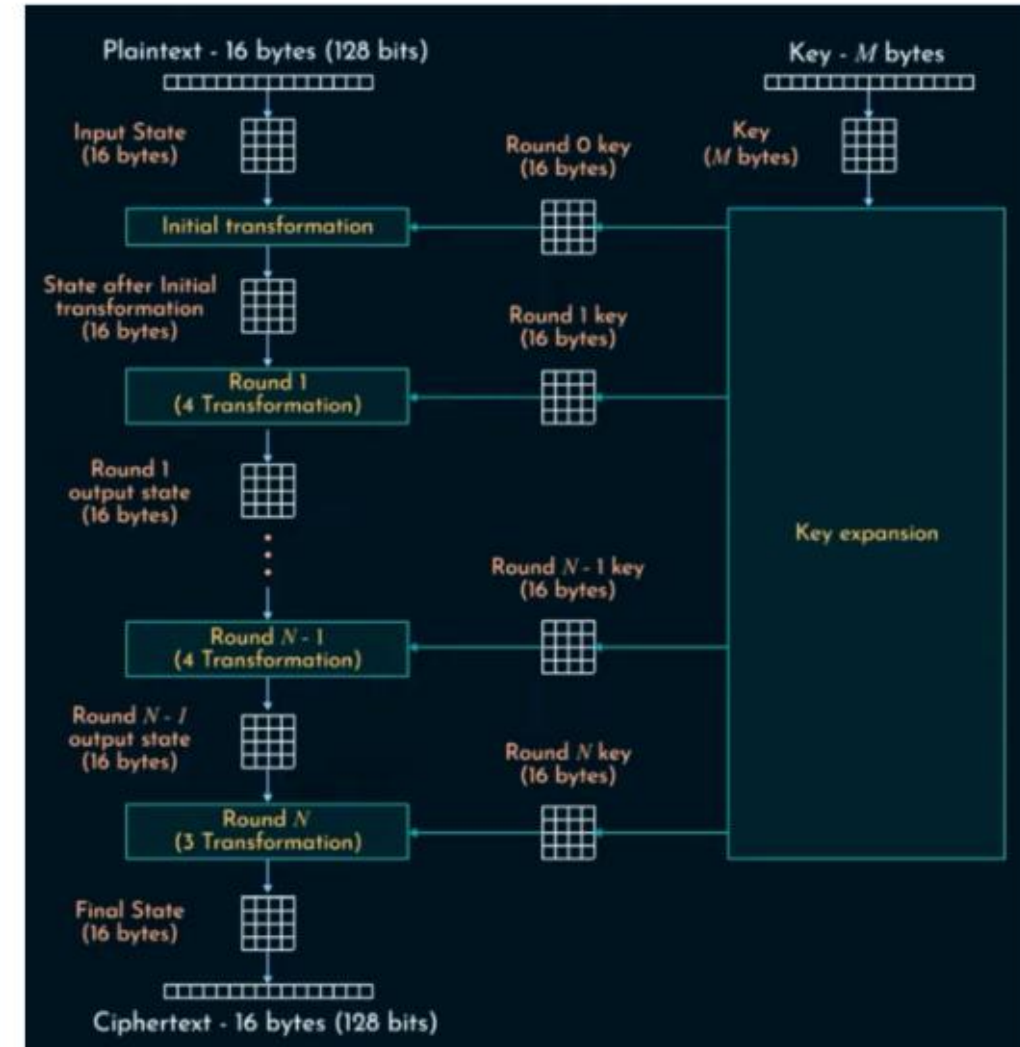
- For the practical exercises P1-P5, we will focus on AES encryption and decryption algorithms
- Implementation on Ultra-96 board, involving different types of MCUs and/or HW accelerators



Source: Techtargat.com (accessed on September 20, 2023).

AES in the FPGA: Why?

- simple operations applied to 128 Bits of data
- Multiple rounds of the same set of operations
 - 10 rounds for 128 bit key
 - 12 rounds for 192 bit key
 - 14 rounds for 256 bit key
- Per round:
 - Substitute Bytes (logic operation)
 - Shift Rows (data reordering)
 - Mix Columns (data reordering)
 - Add Round Key (logic operation)
- AES is designed to be fast, yet hard to crack
- AES explained:
<https://www.youtube.com/watch?v=O4xNJsjtN6E>
<https://www.youtube.com/watch?v=gP4PqVGudtg>



FPGA

XCZU3EG-1SFVC784I

Processors

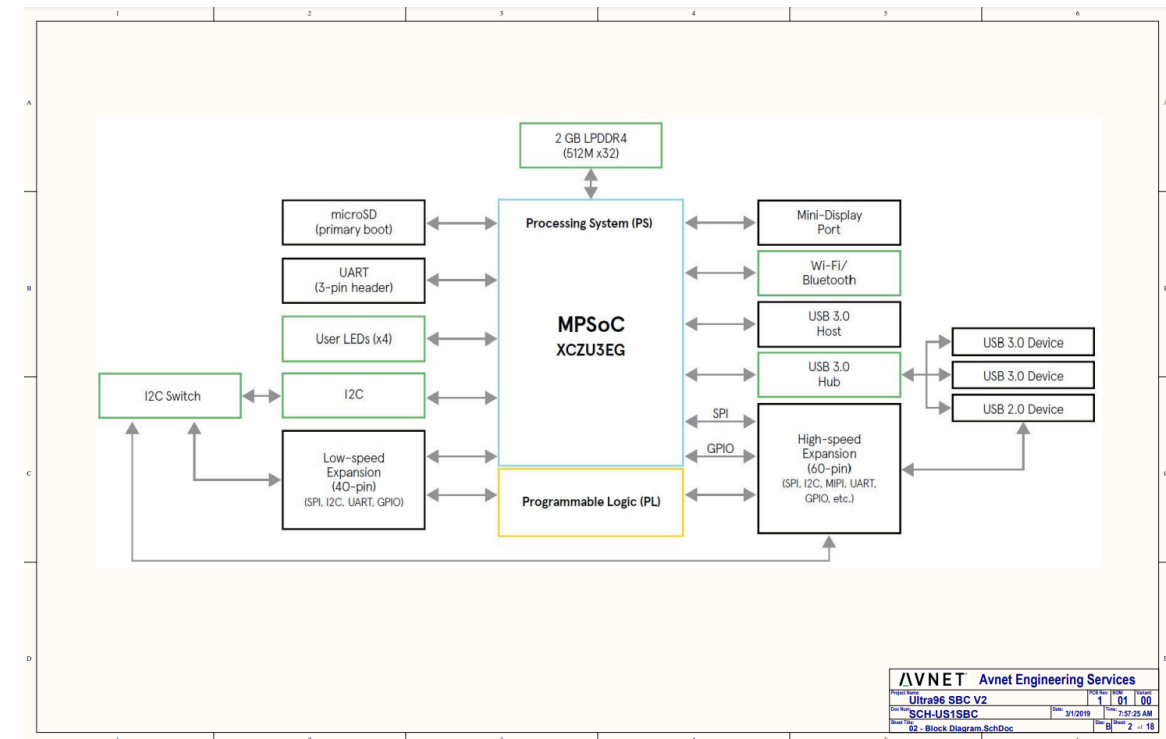
4x ARM Cortex-A53,
1.2GHz
2x ARM Cortex-R5,
500MHz

GPU

ARM Mali-400 MP2

RAM

LPDDR4, 2GB, 32bit Bus,
1066 Mbps



Design Space for Embedded Systems

■ HW: Options to choose from:

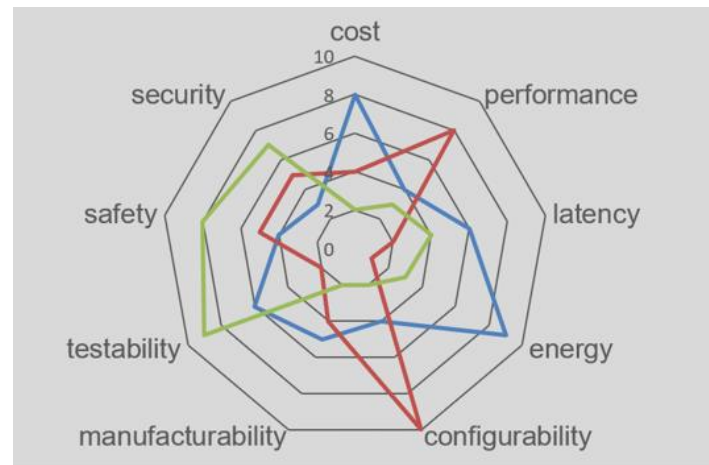
- Type of MCU (e.g., ARM-M0+, ARM-R5, RISC-V, TI MSP430, etc.)
- Number of MCUs (1, 2, ...)
- Cache System (size, type, # levels)
- Hardware Accelerators (special purpose, FPGA)
- Interconnect between MCUs (bus systems, bus widths)
- Sensor Types (analog vs. digital sensor)
- ...

■ SW: We have the flexibility...

- ... of binding SW Blocks on different HW components
- ... of using operating system support
- ... of coding bare-metal applications

Embedded System Design – Optimization Problem

- Essentially, the design of an Embedded System is an optimization problem with more than one objective
- In the past weeks, we discussed several examples of objectives for Embedded System design



- An optimization problem with several objectives is called a «multiobjective optimization problem»

Comparison of Designs

- Usually, for multiobjective optimization problems, there is not just a “best” solution
- Example: “Notebook”
 - Possible objectives: Cost, Performance
 - Which notebook is better?

350.-
Lenovo V15 Gen 4
15.60", Intel Core i3-13150L, 8 GB, 256 GB, CH

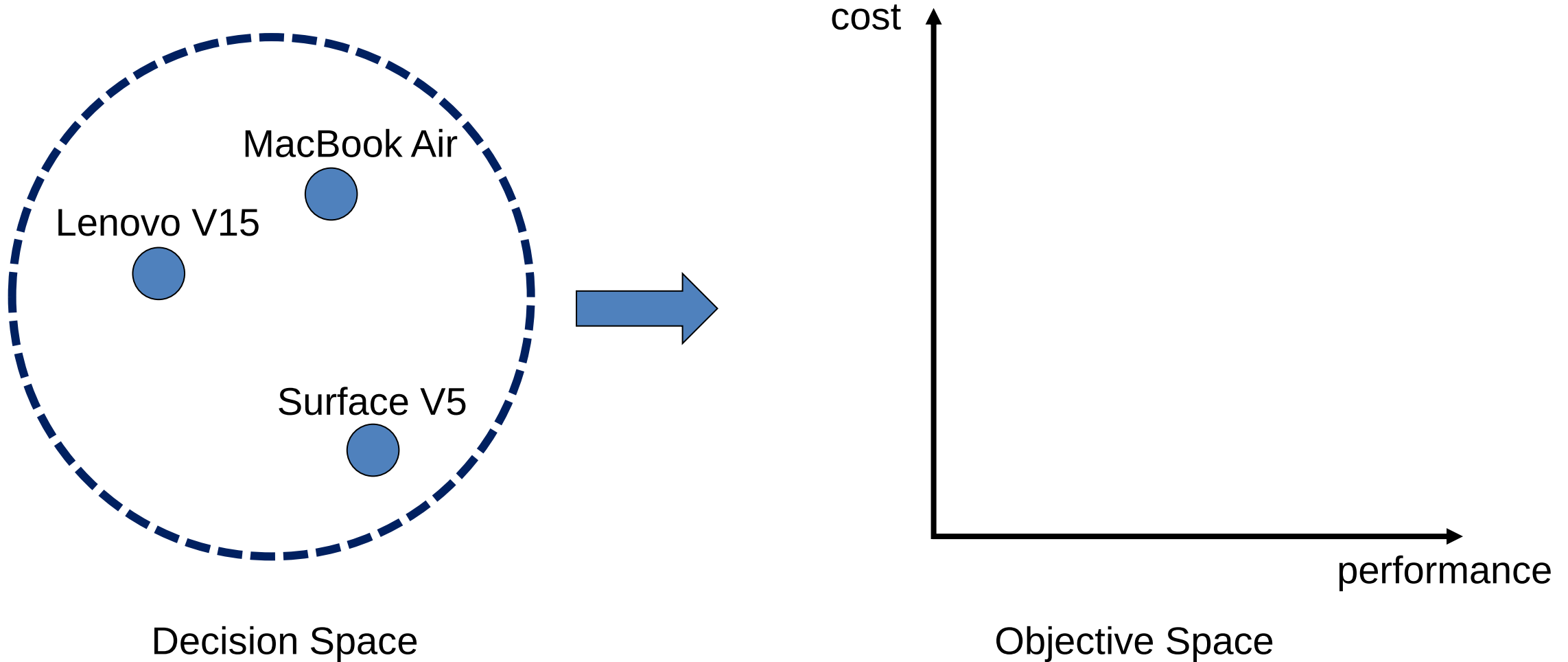


intel

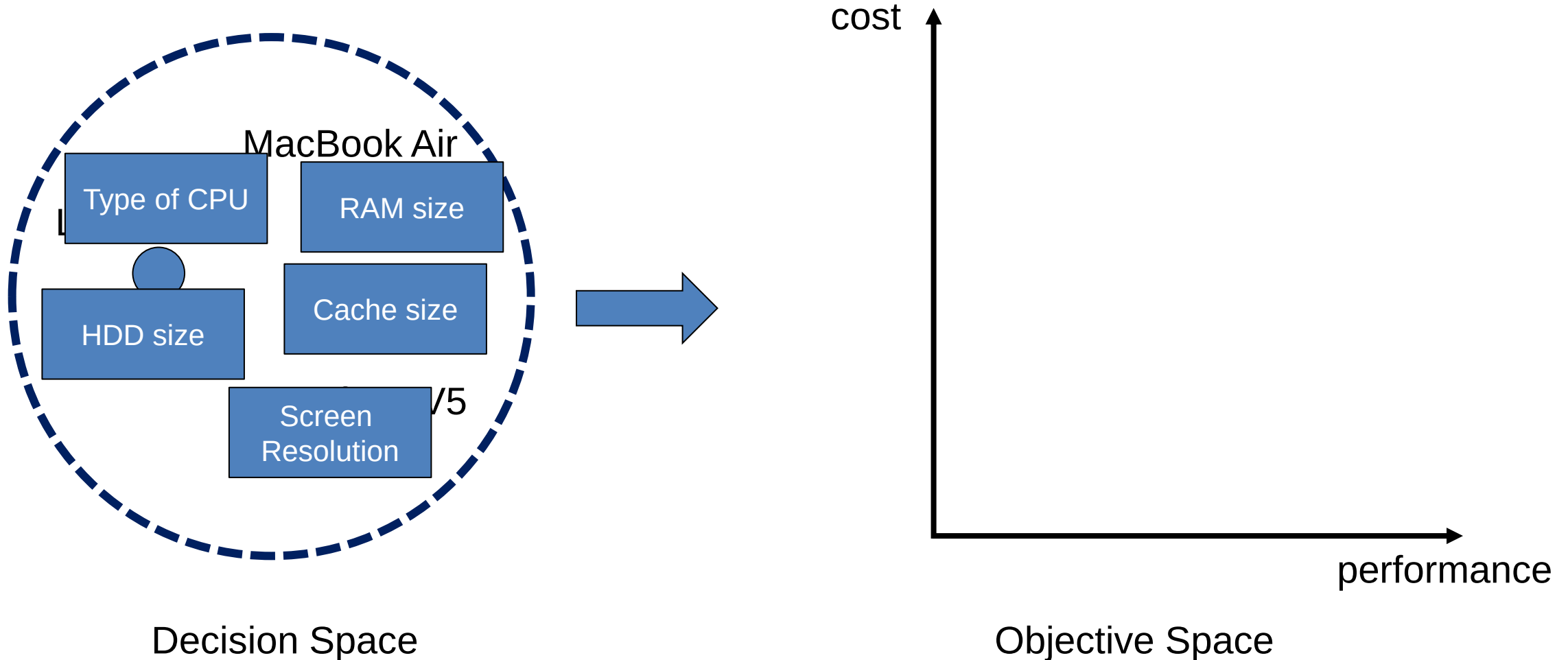
2302.-
Microsoft Surface Laptop 5 for Business
15", Intel Core i7-12650L, 32 GB, 1000 GB, CH



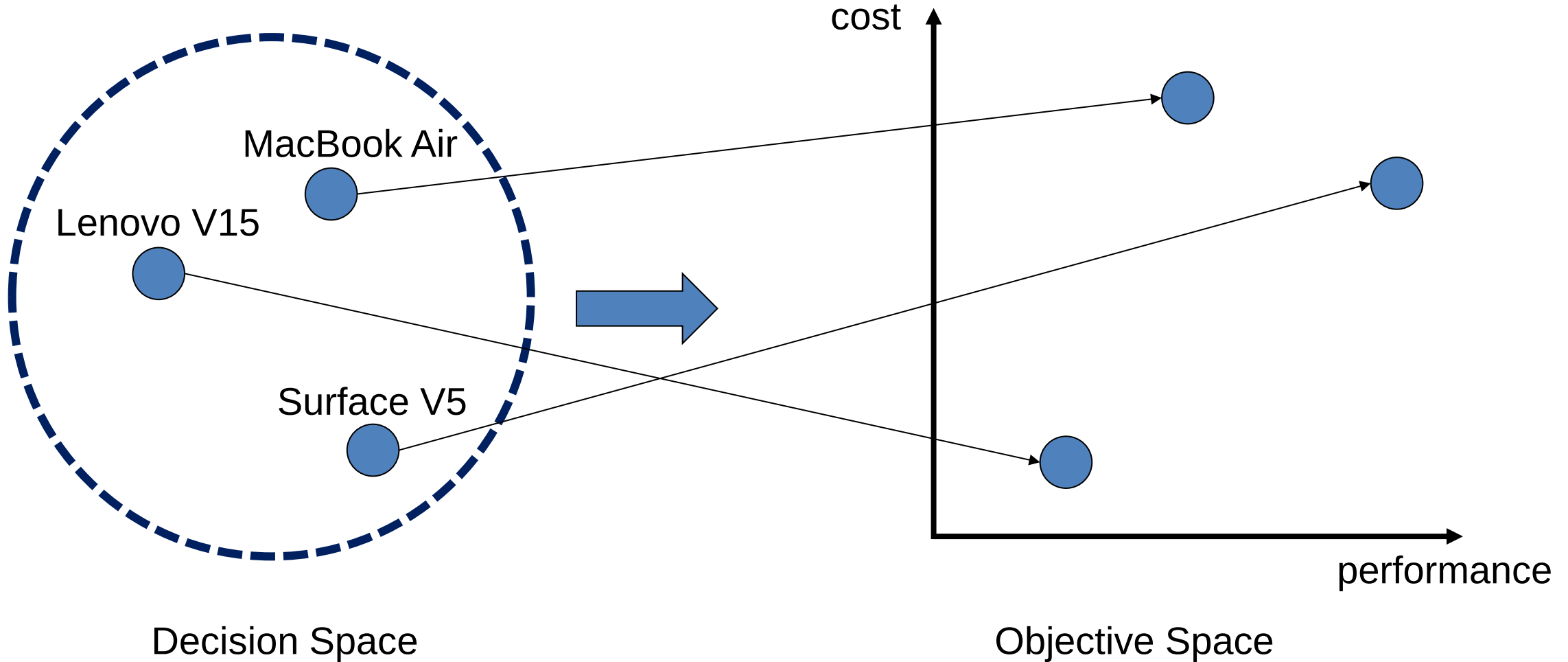
Decision Space vs. Objective Space



Decision Space vs. Objective Space



Decision Space vs. Objective Space

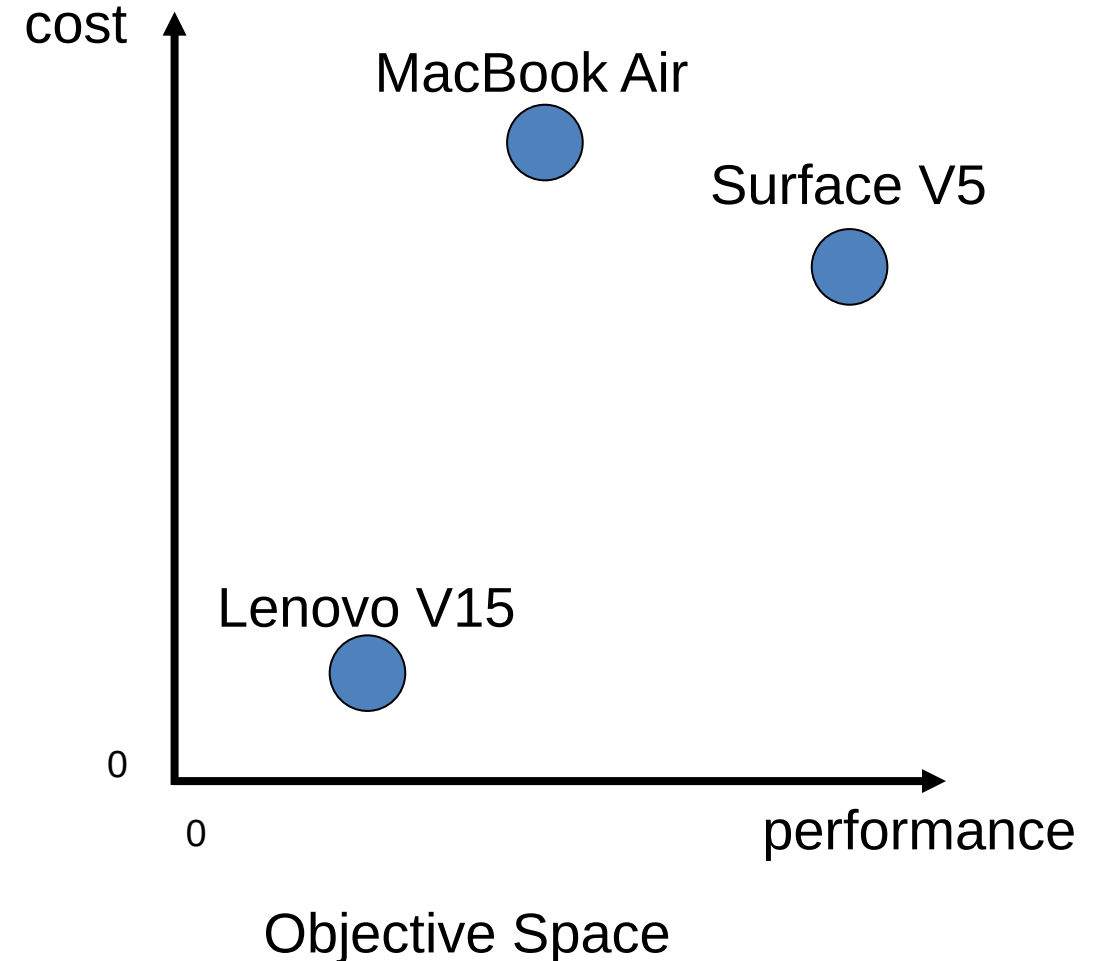


Pareto-Dominance

■ Definition:

- A solution x weakly dominates a solution y if it is at least as good in all objectives $f_i(x) \leq f_i(y)$ for all $1 \leq i \leq n$ for n objectives.
- The solution x is better than y if x weakly dominates y AND $f_i(x) < f_i(y)$ for at least some i .

Remark: The smaller the objective value $f_i(x)$, the better.

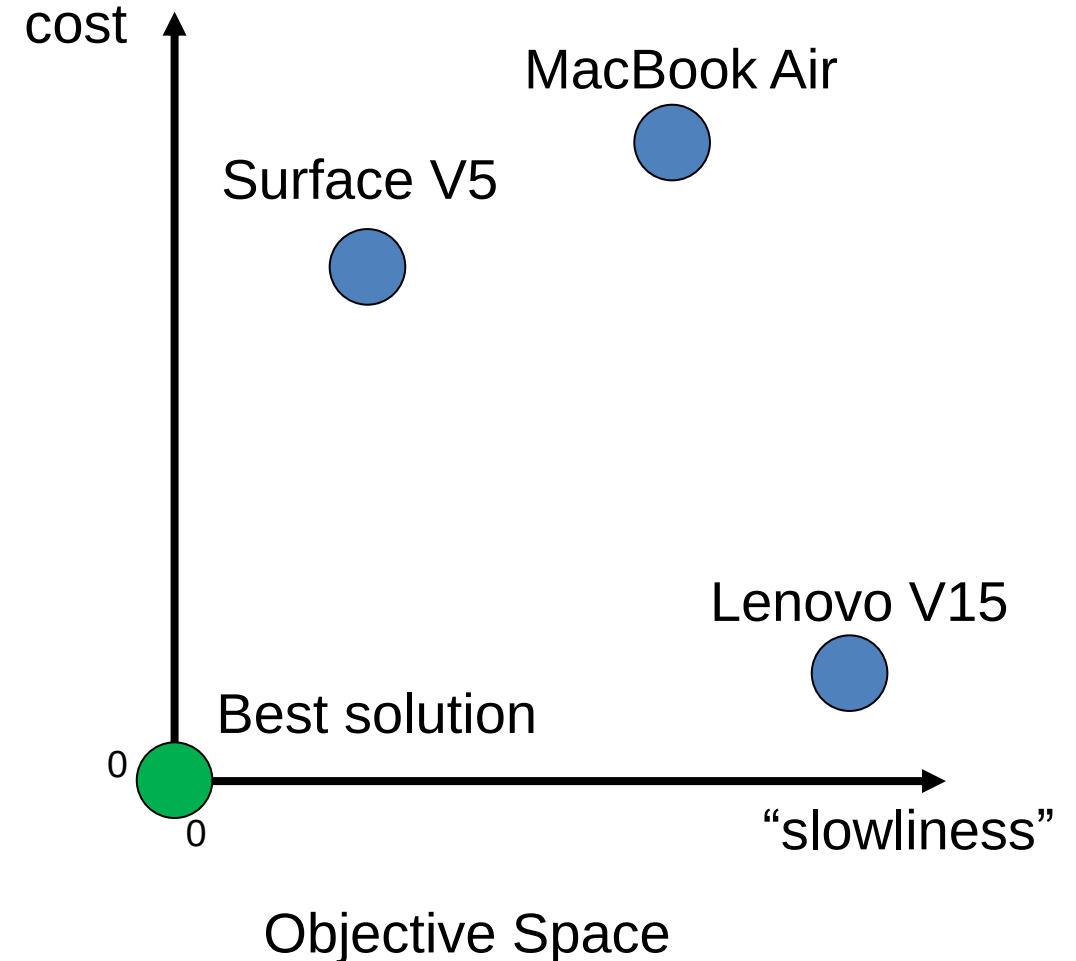


Pareto-Dominance

■ Definition:

- A solution x weakly dominates a solution y if it is at least as good in all objectives $f_i(x) \leq f_i(y)$ for all $1 \leq i \leq n$ for n objectives.
- The solution x is better than y if x weakly dominates y AND $f_i(x) < f_i(y)$ for at least some i .

Remark: The smaller the objective value $f_i(x)$, the better.

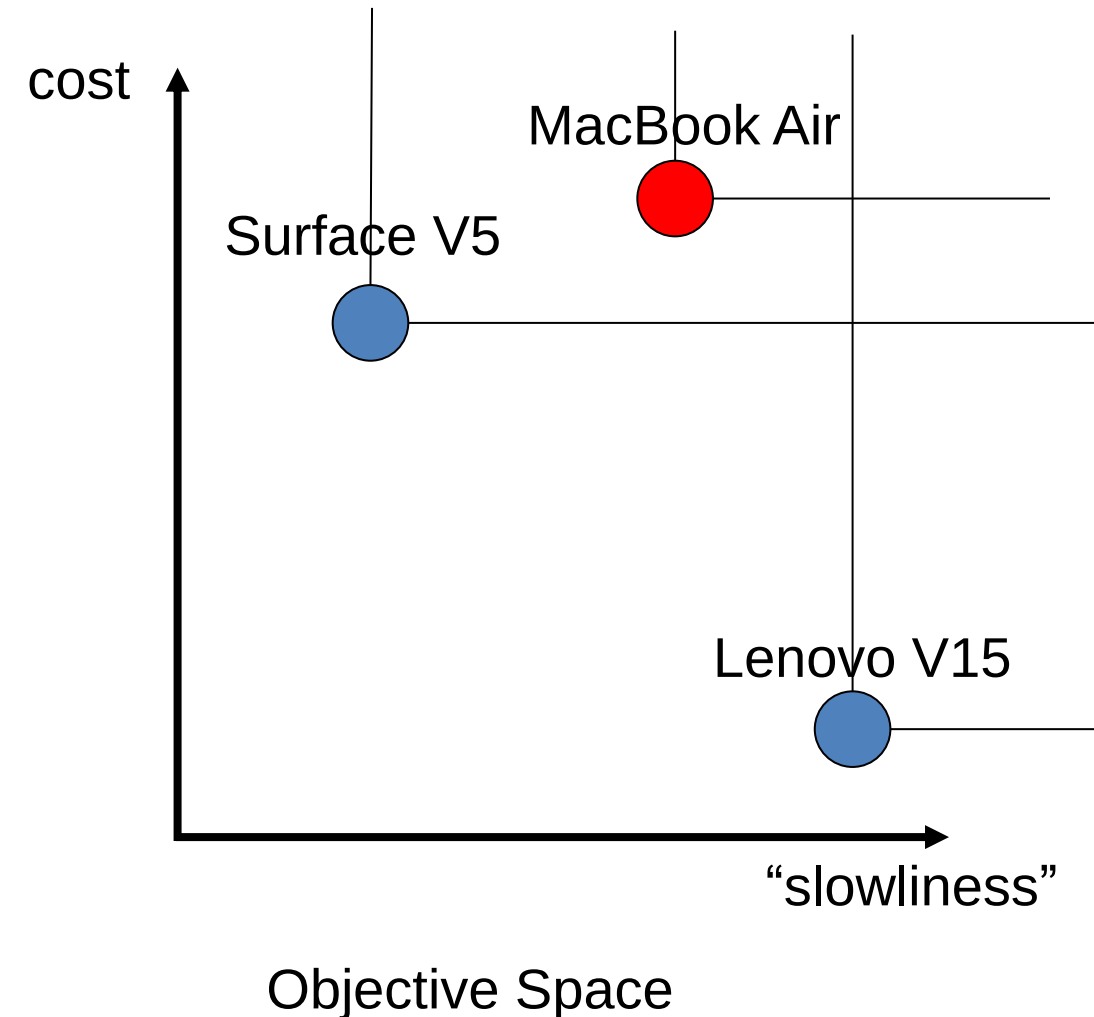


Pareto-Dominance

■ Definition:

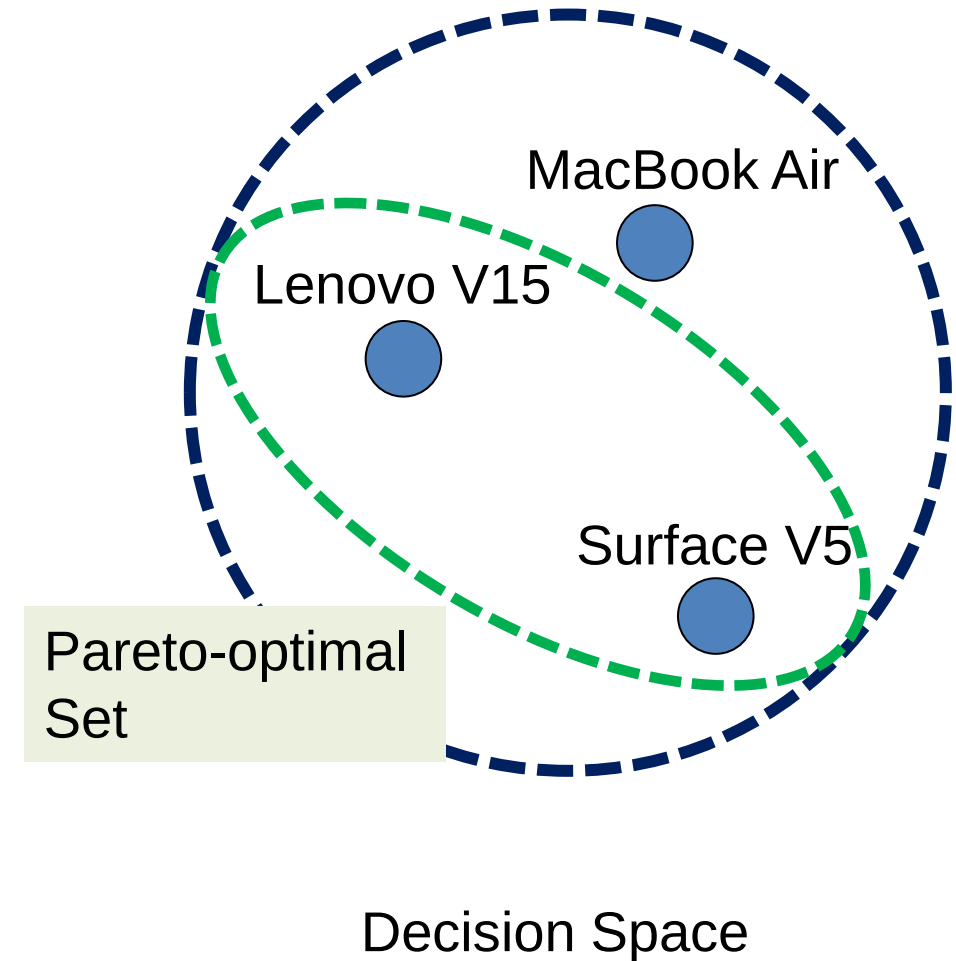
- A solution x weakly dominates a solution y if it is at least as good in all objectives $f_i(x) \leq f_i(y)$ for all $1 \leq i \leq n$ for n objectives.
- The solution x is better than y if x weakly dominates y AND $f_i(x) < f_i(y)$ for at least some i .

Remark: The smaller the objective value $f_i(x)$, the better.



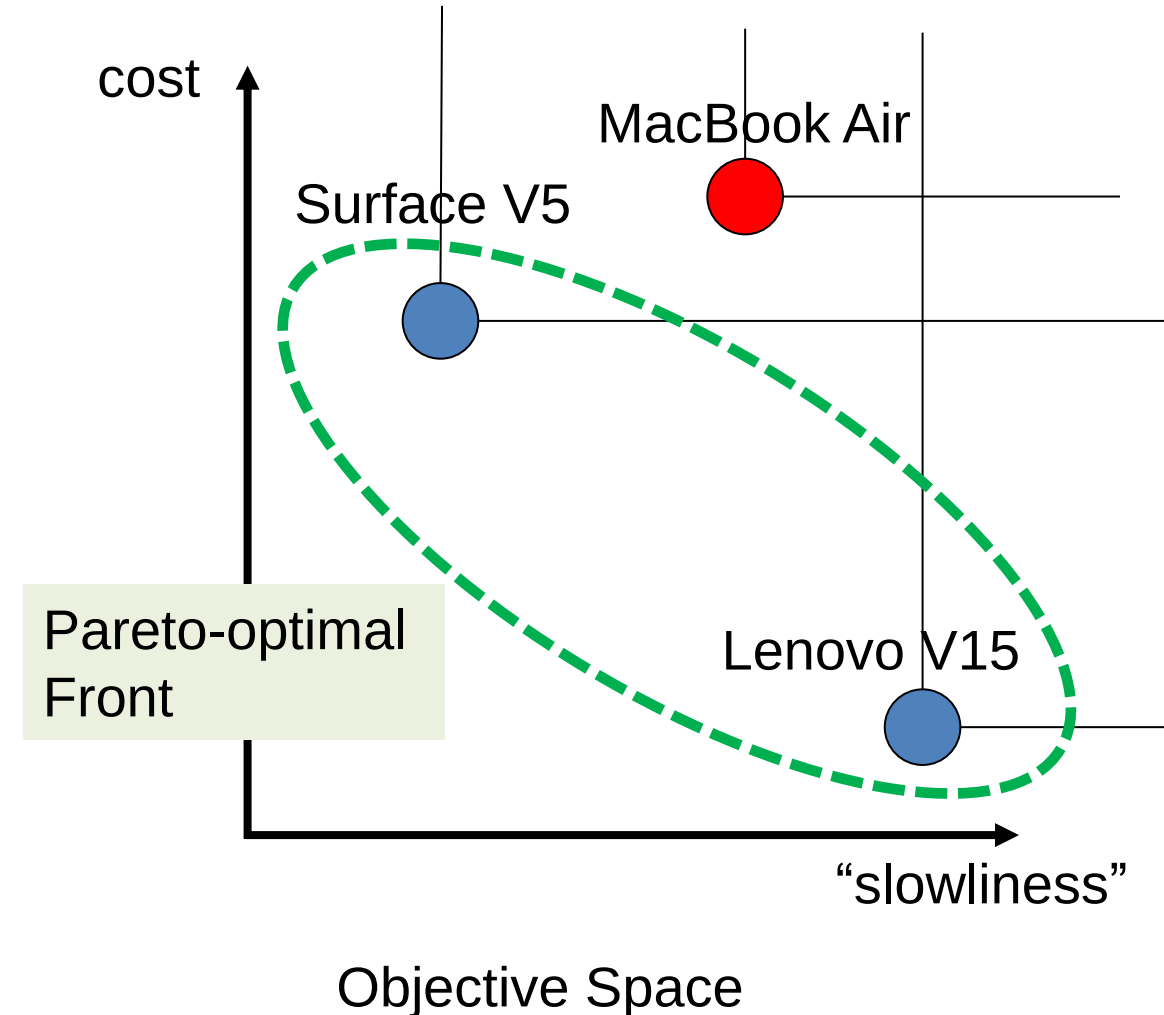
Pareto-optimal Set

- A solution x is named «Pareto-optimal» if it is not Pareto-dominated by any other solution in the decision space X
- The set of all Pareto-optimal solutions is called the «Pareto-optimal Set»
- The image in the objective space of the Pareto-optimal set is named «Pareto-optimal Front»

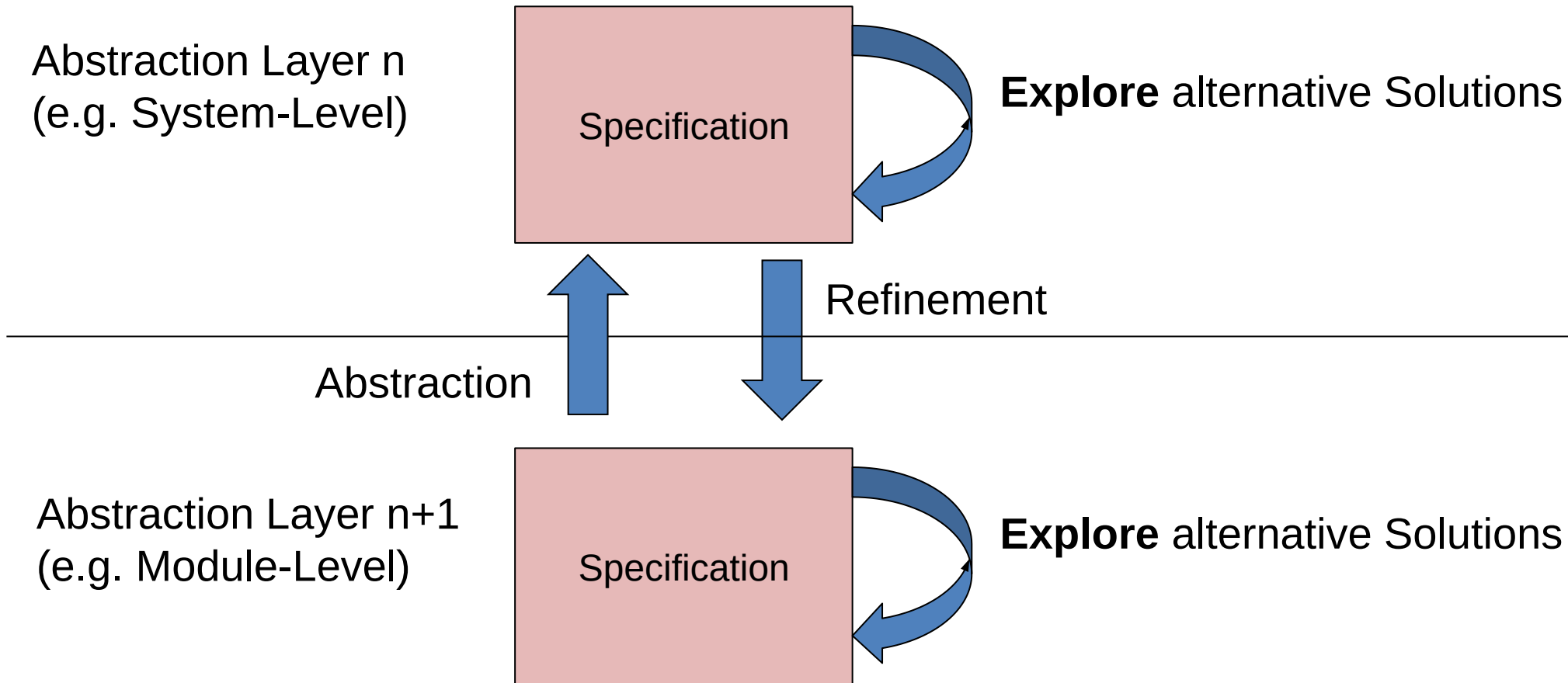


Pareto-optimal Front

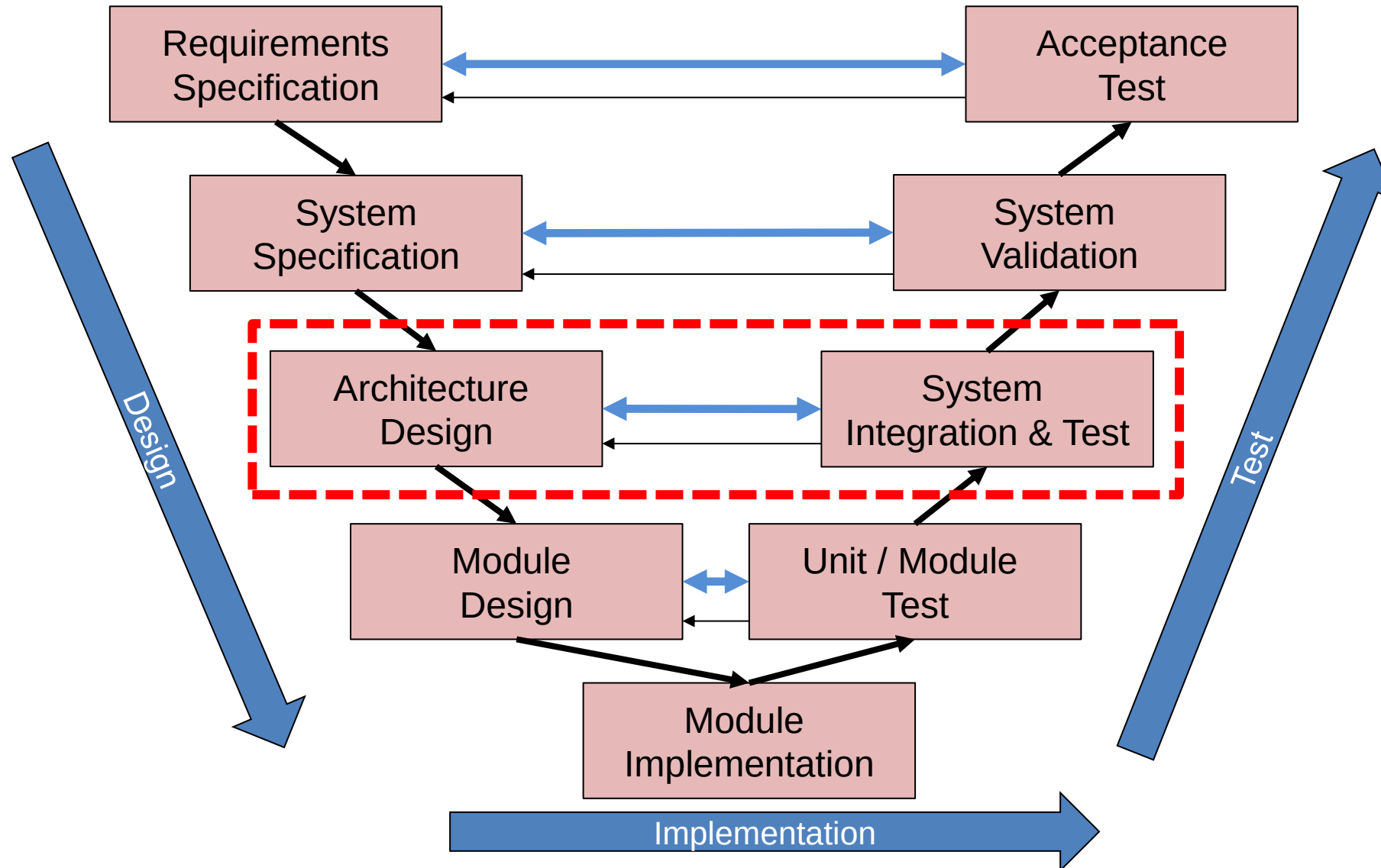
- A solution x is named «Pareto-optimal» if it is not Pareto-dominated by any other solution in the decision space X
- The set of all Pareto-optimal solutions is called the «Pareto-optimal Set»
- The image in the objective space of the Pareto-optimal set is named «Pareto-optimal Front»



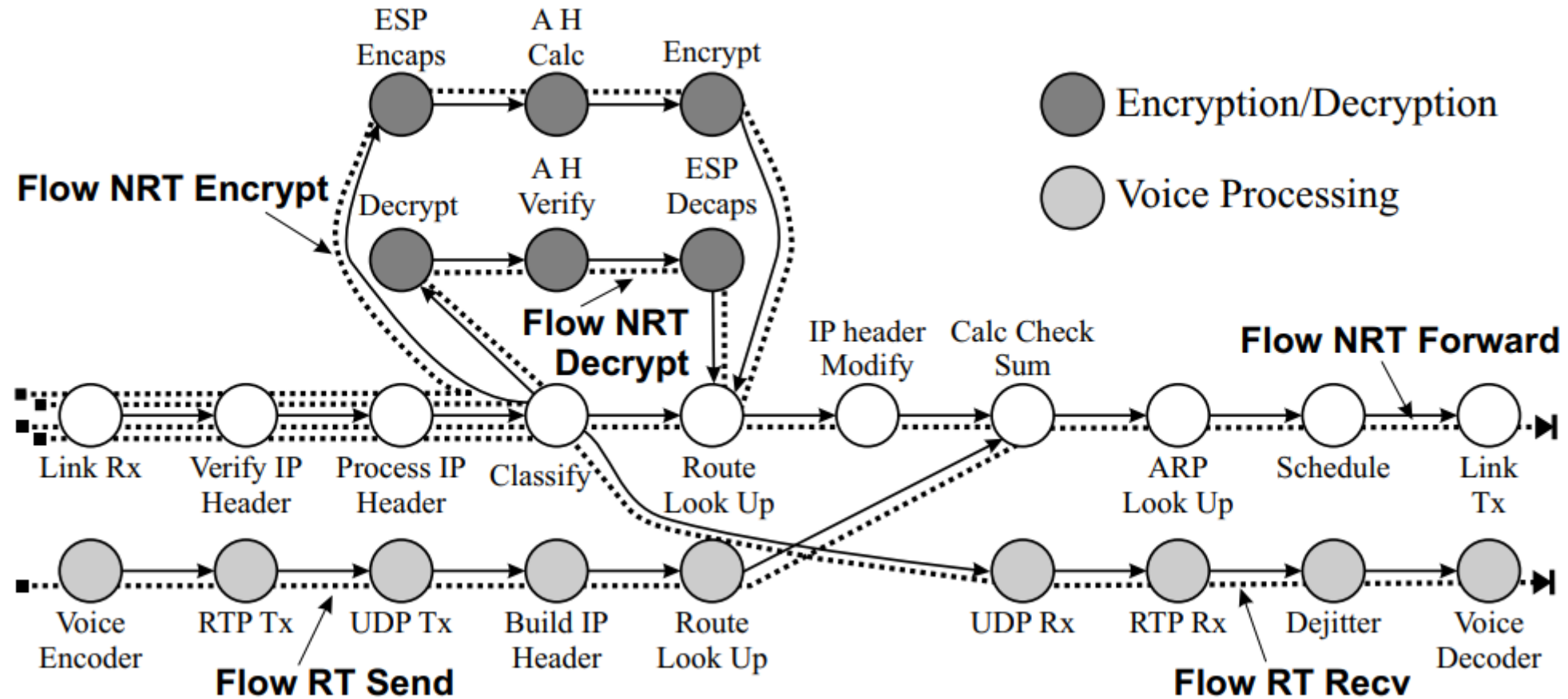
Specification, **Exploration**, Refinement



V -Model

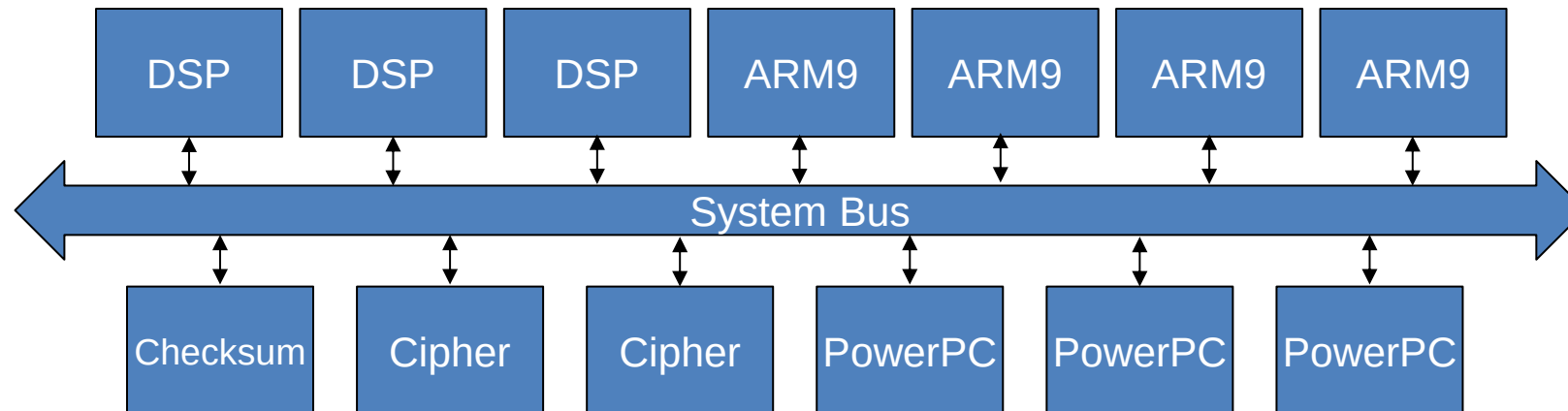


Case Study 1: Network Processor



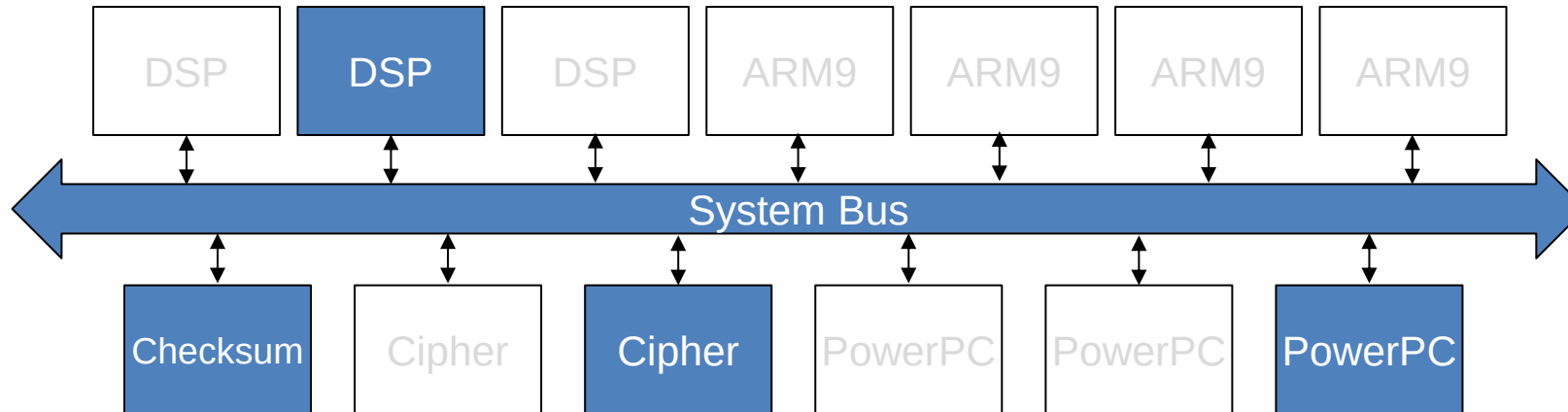
Case Study 1: Network Processor → Design Space

- **Architecture Template (possible HW elements)**



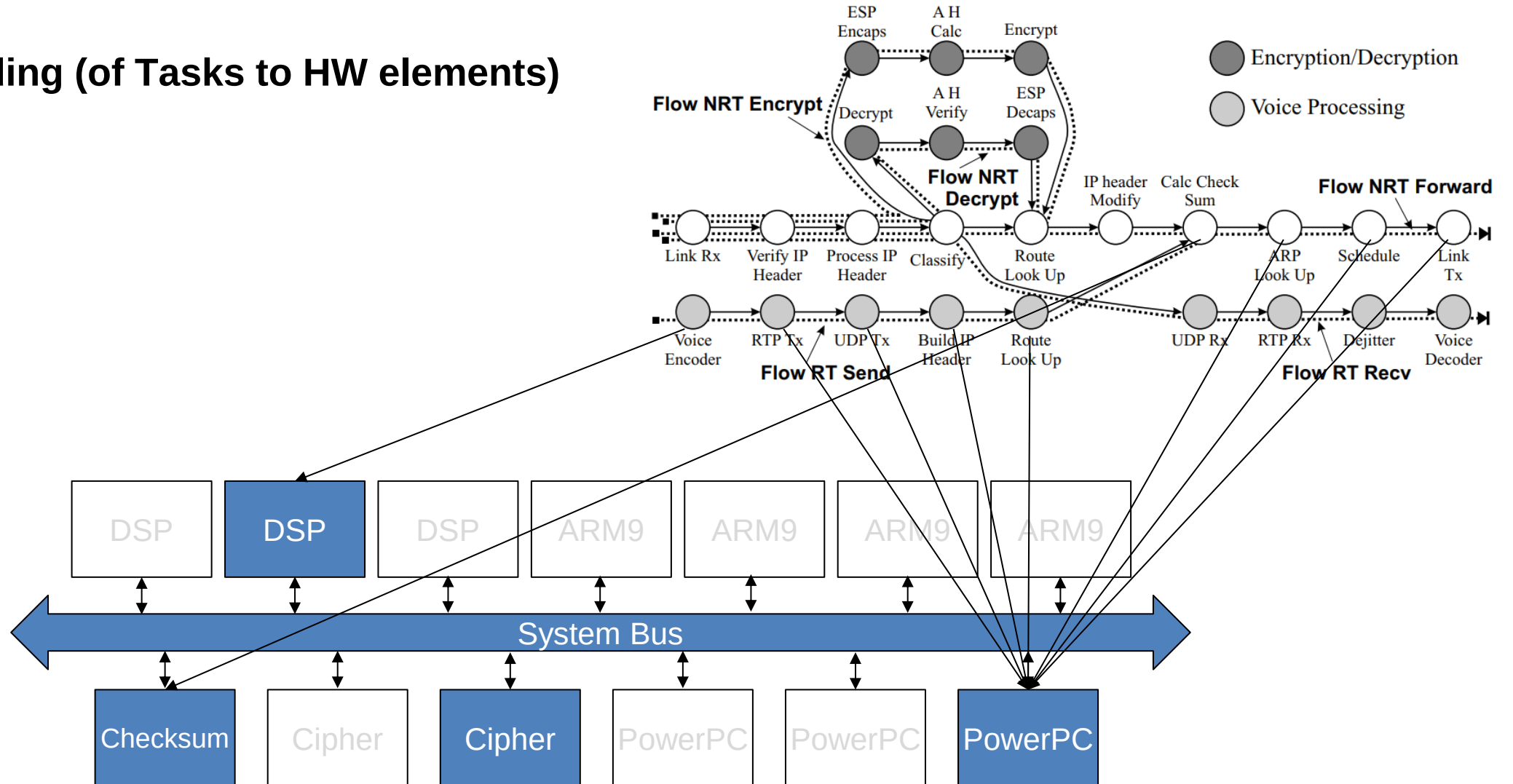
Case Study 1: Network Processor → Design Space

■ Allocation (selection of HW elements)

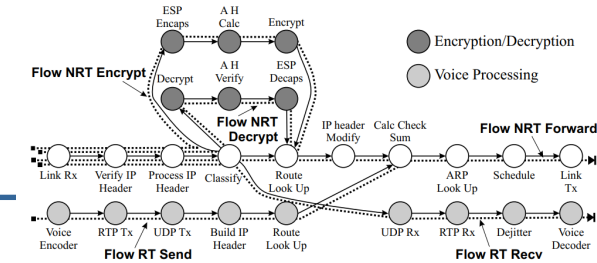


Case Study 1: Network Processor → Design Space

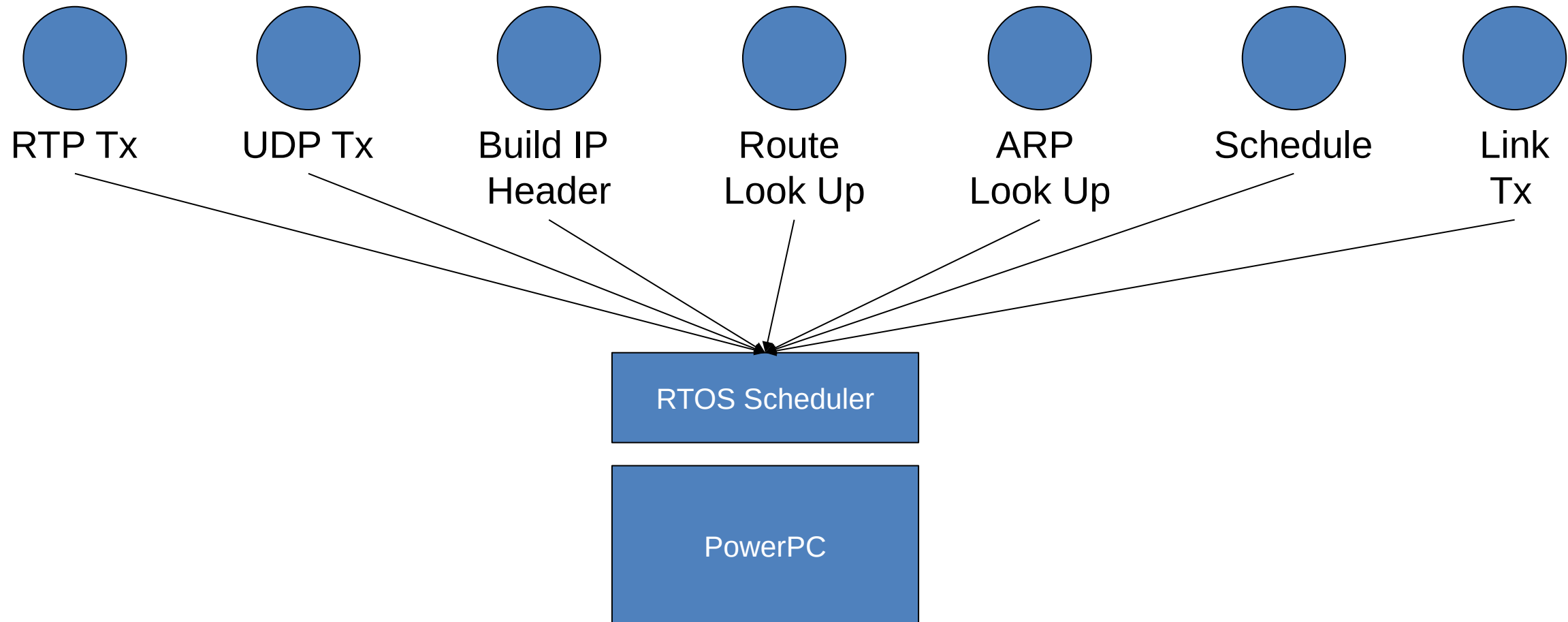
■ Binding (of Tasks to HW elements)



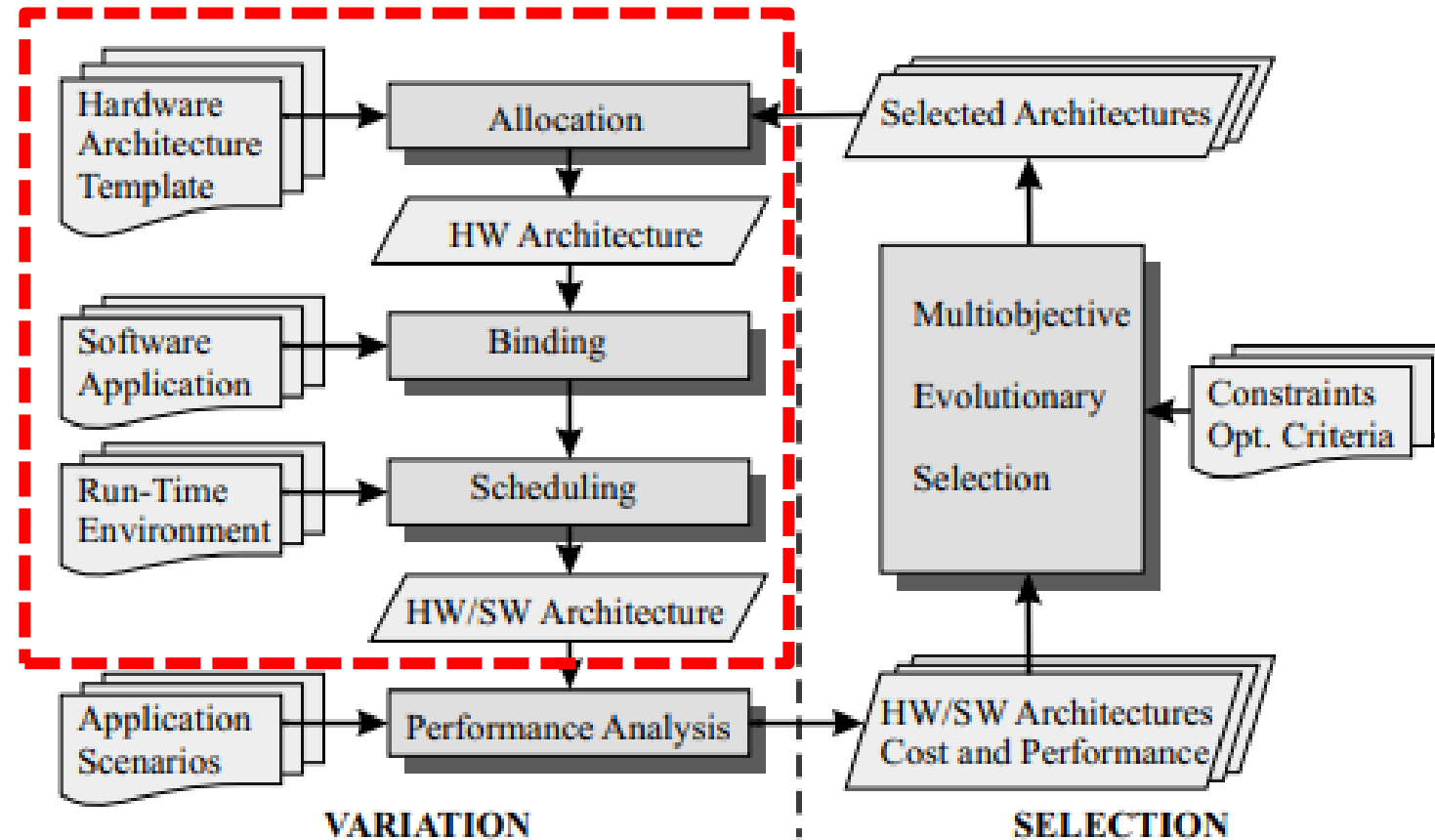
Case Study 1: Network Processor → Design Space



■ Runtime Environment (RTOS with Scheduling)



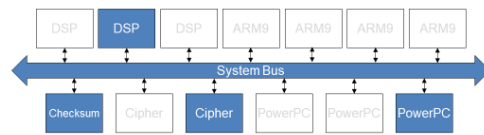
Case Study 1: Network Processor → Summary



Case Study 1: Network Processor → Performance Analysis

Case Study 1: Network Processor → Design Space

Allocation (selection of HW elements)



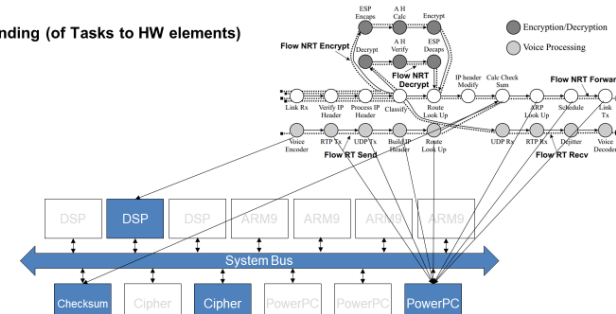
HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

26

Case Study 1: Network Processor → Design Space

Binding (of Tasks to HW elements)



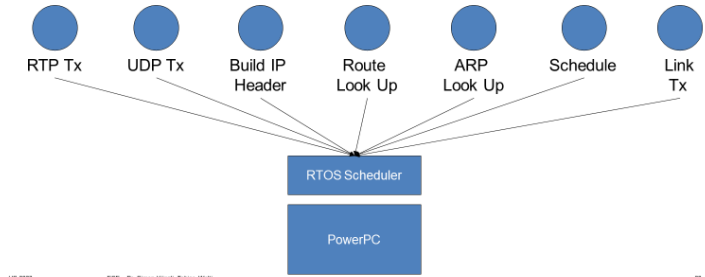
HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

27

Case Study 1: Network Processor → Design Space

Runtime Environment (RTOS with Scheduling)



HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

28

Component Cost Σ

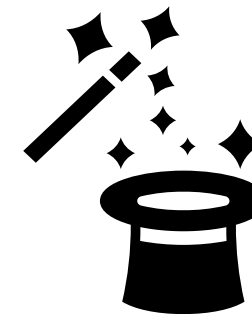
Objective Value 1

Performance Analysis
for
Access Network

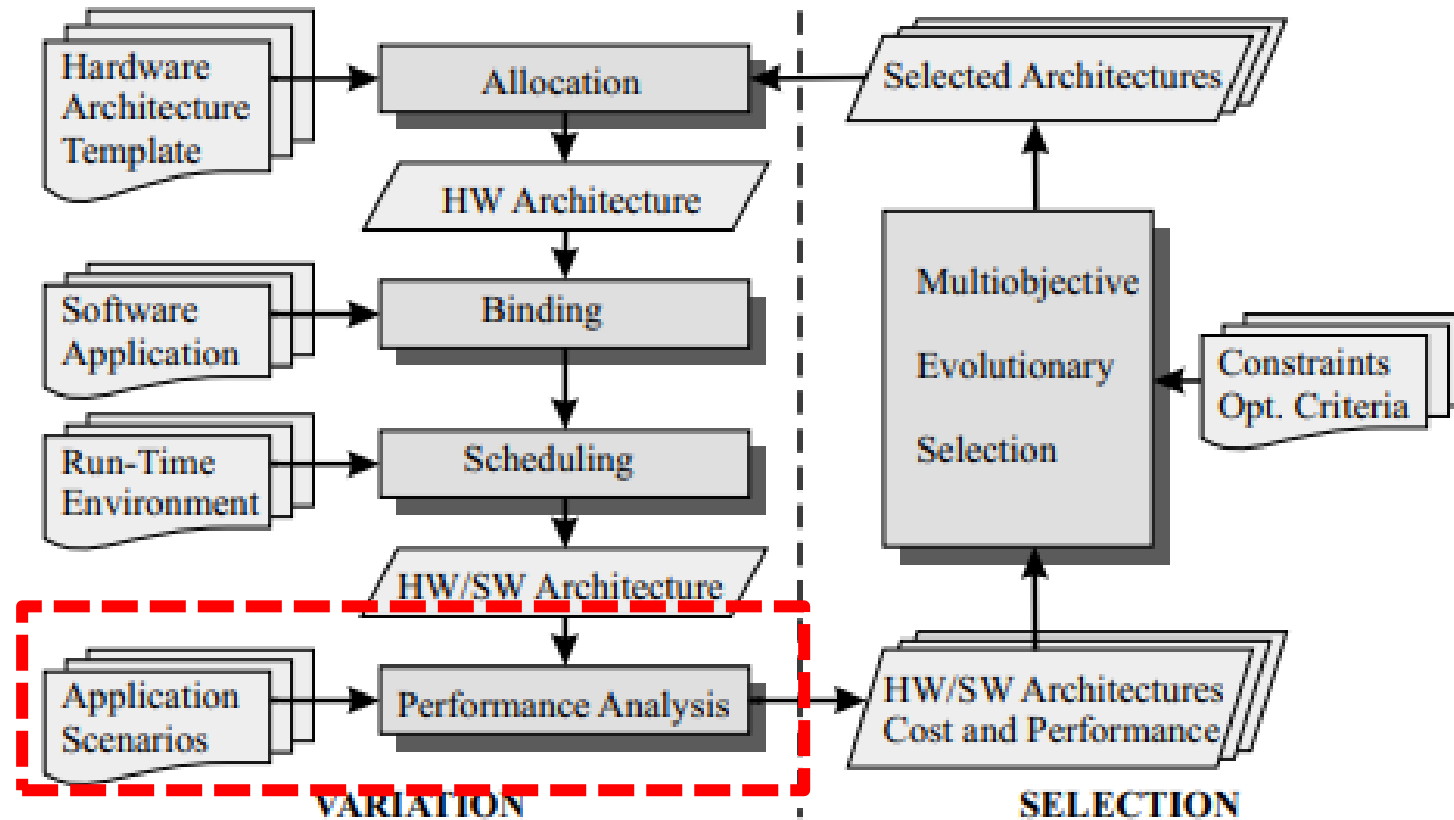
Objective Value 2

Performance Analysis
for
Backbone Network

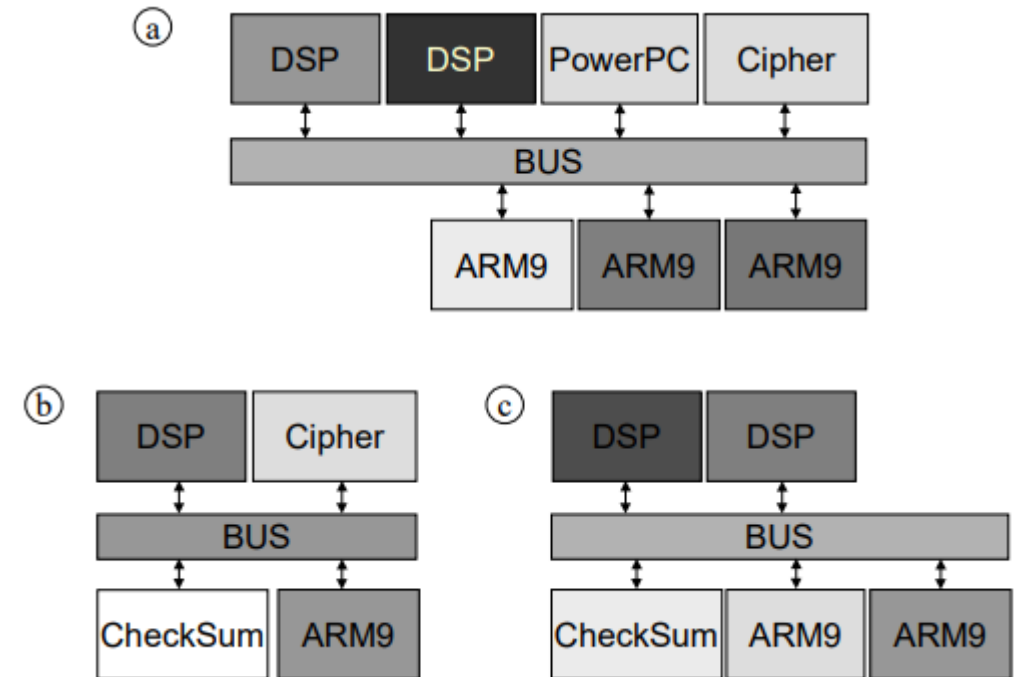
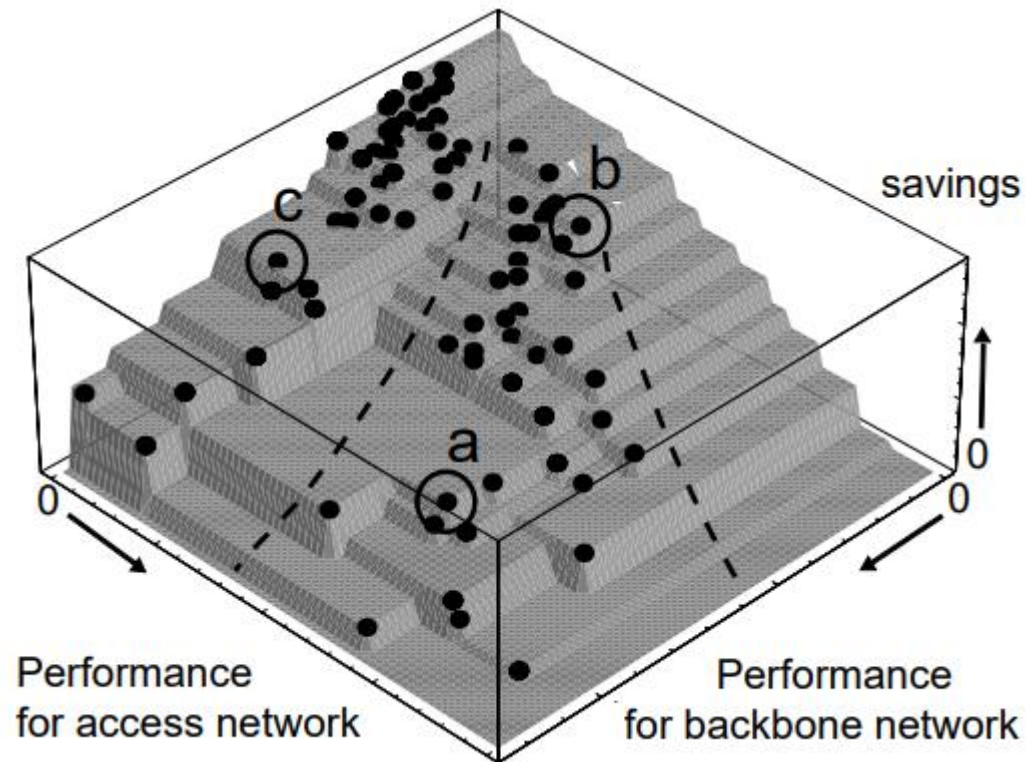
Objective Value 3



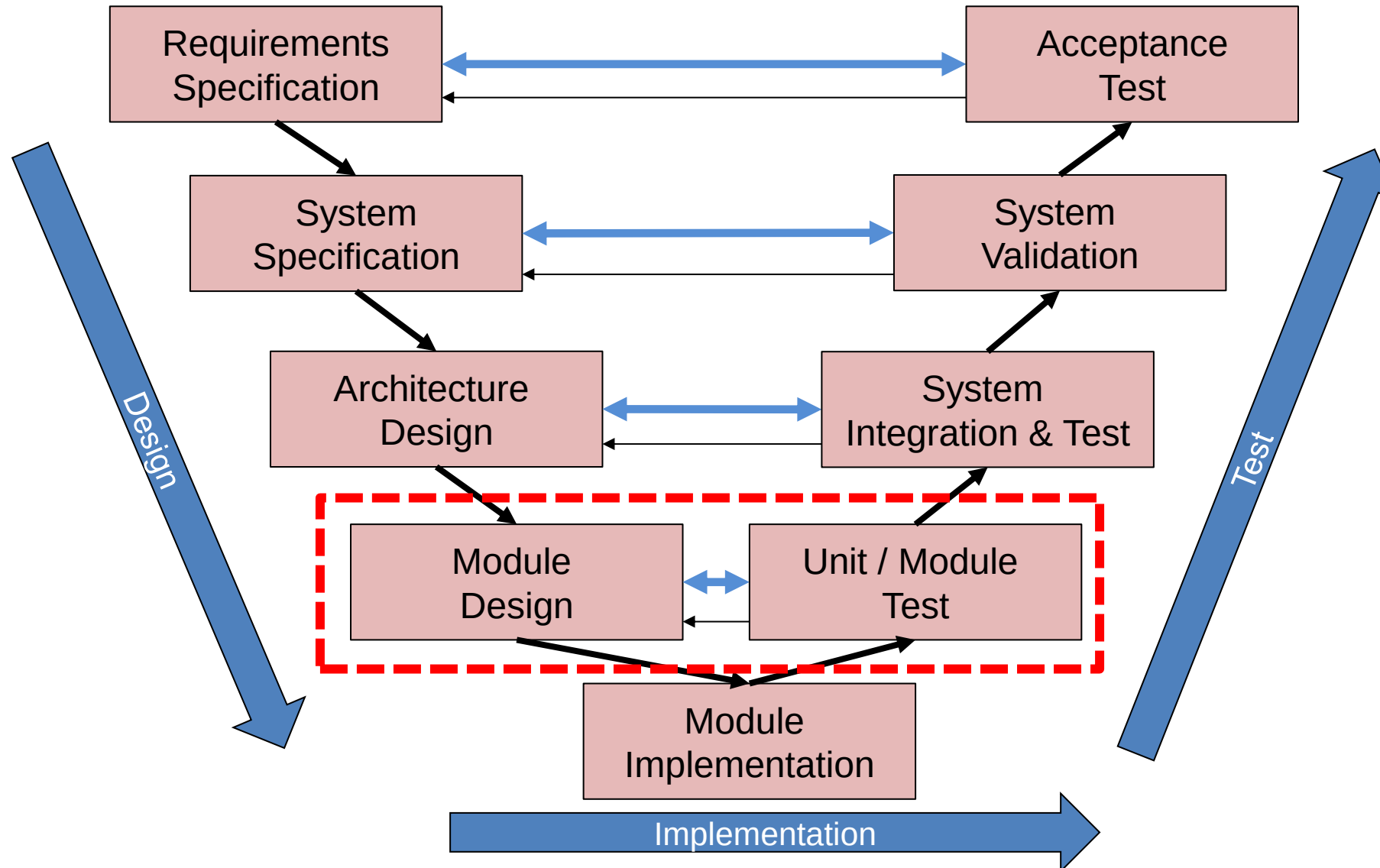
Case Study 1: Network Processor → Summary



Case Study 1: Network Processor → Solutions

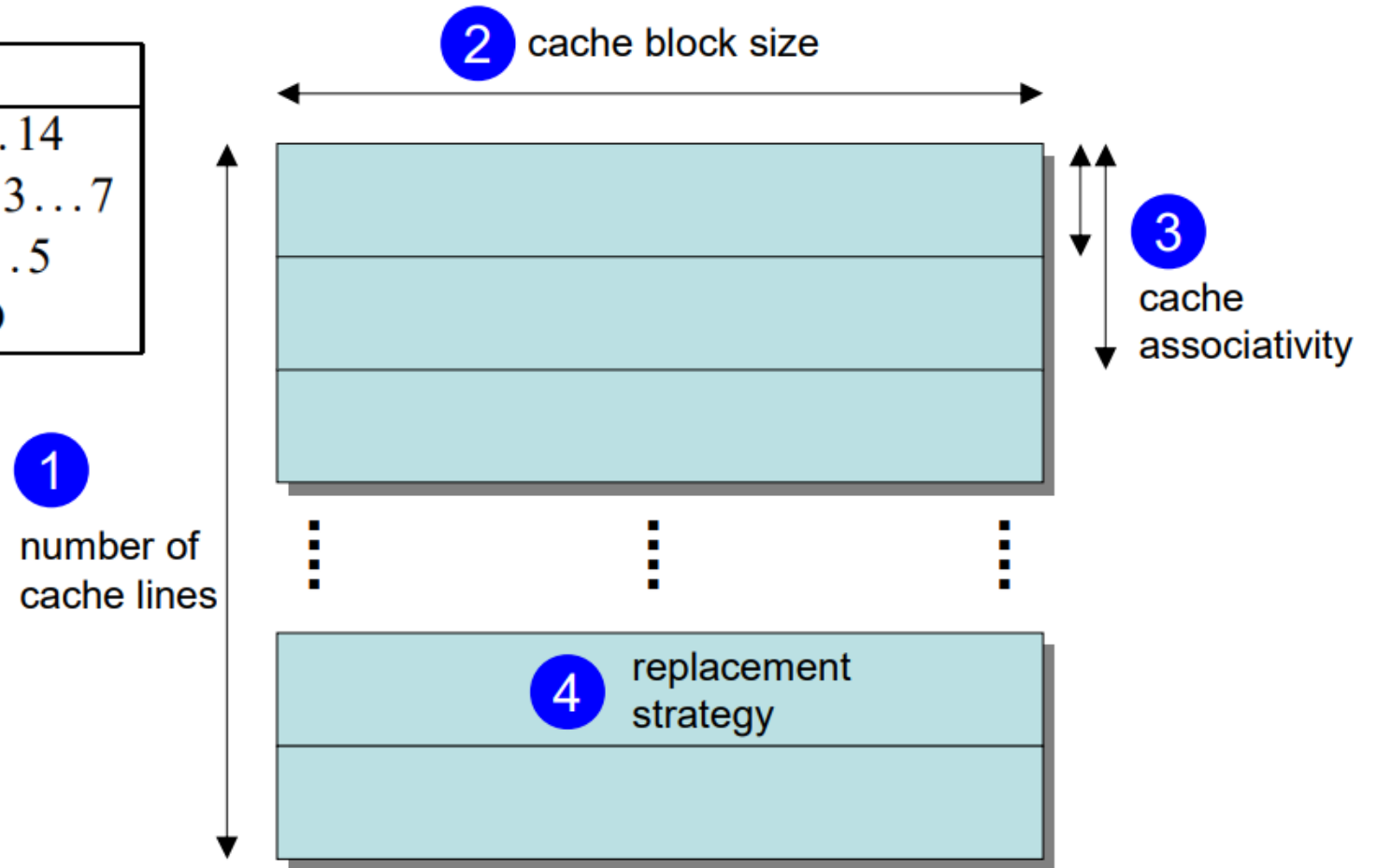


V -Model



Case Study 2: Cache Organization → Design Space

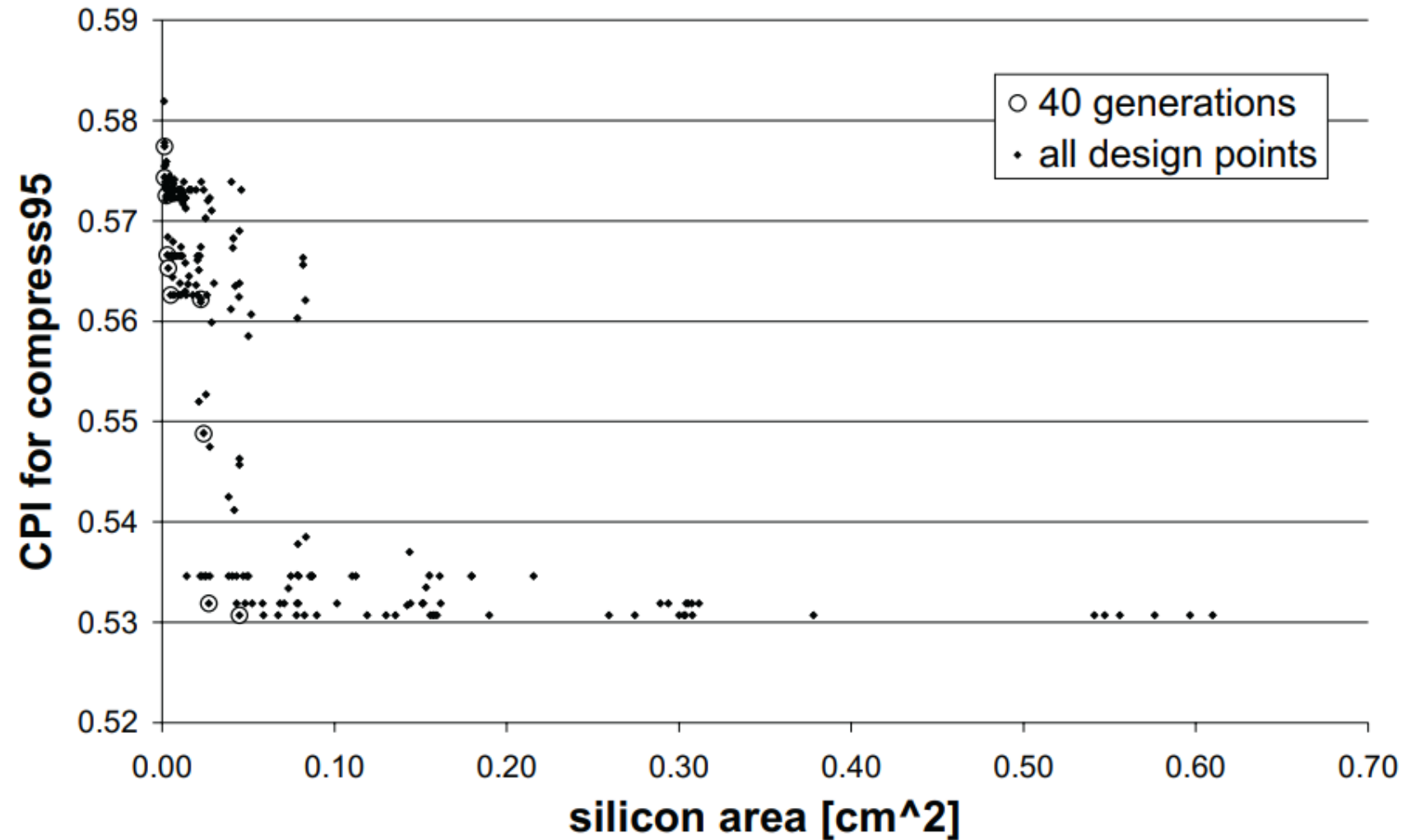
Parameter	Range
# of Cache lines	2^k , with $k = 6 \dots 14$
Block size	2^k Bytes, with $k = 3 \dots 7$
Associativity	2^k , with $k = 0 \dots 5$
Replacement strategy	LRU or FIFO



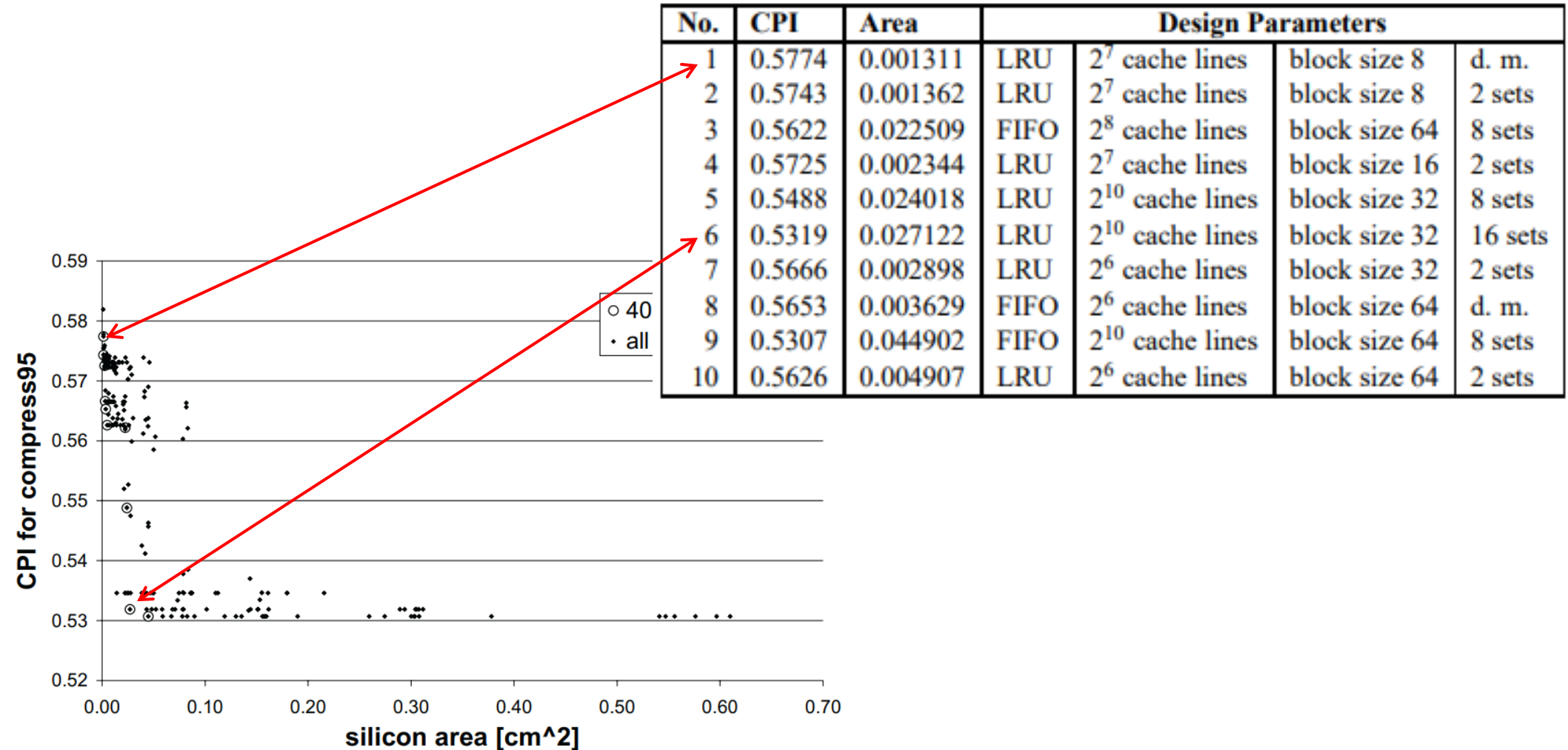
Case Study 2: Cache Organization → Objective Space

The first objective according to which the cache parameters are to be optimised is the CPI (cycles per instruction) achieved for a sample benchmark application and the second objective is the chip area needed to implement the cache on silicon. To estimate the corresponding objective values, two tools were used, namely sim-outorder of SimpleScalar [26] and CACTI [27] provided by Compaq. The first tool served to estimate the CPI for the benchmark compress95 running on the plain text version of the GNU public license as application workload. The smaller the CPI for compress95 for a particular solution, the better is this solution for this objective. The second tool calculated an estimate for the silicon area needed to implement the cache. The smaller the area, the better is the cache for the area objective.

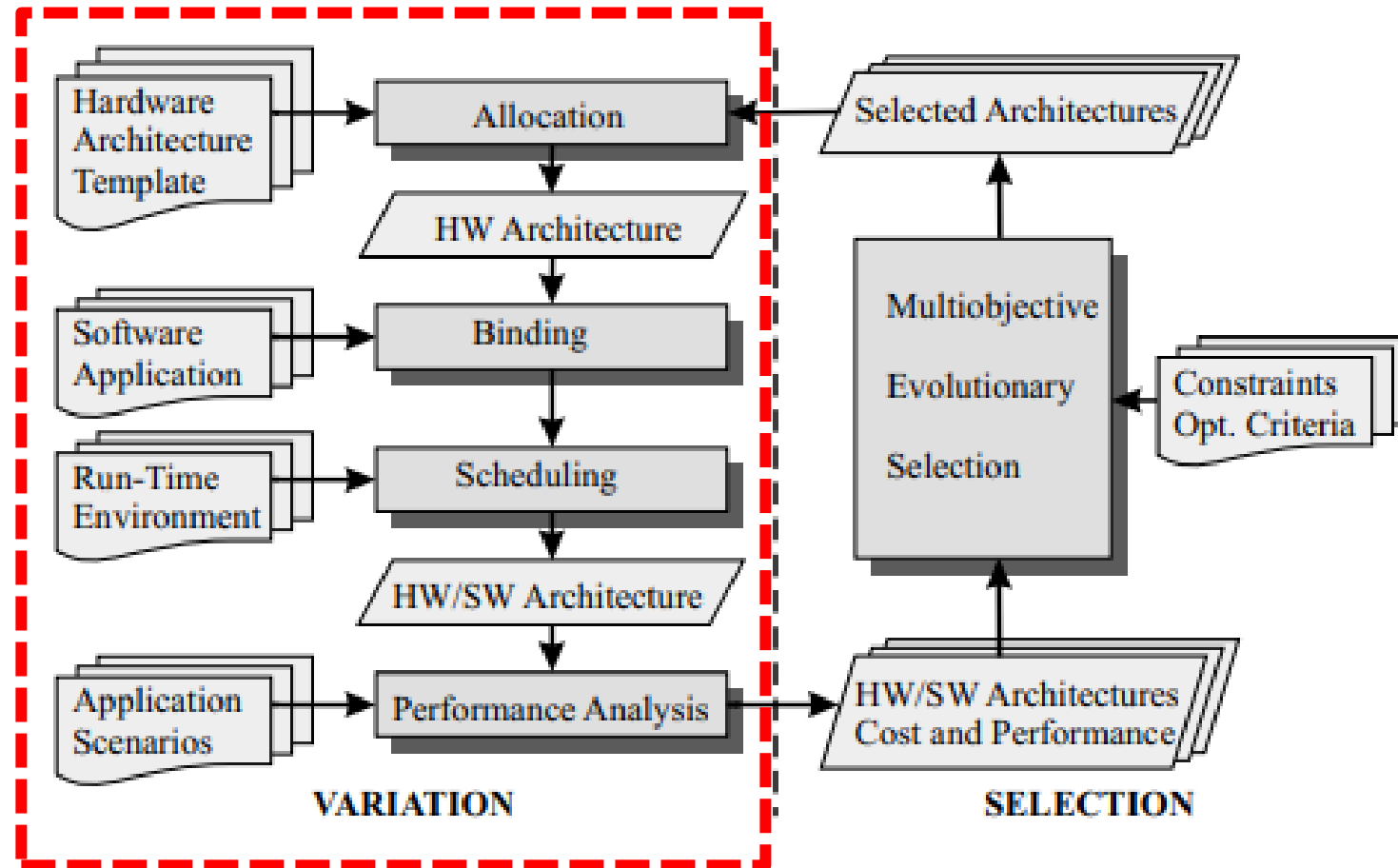
Case Study 2: Cache Organization → Solutions



Case Study 2: Cache Organization

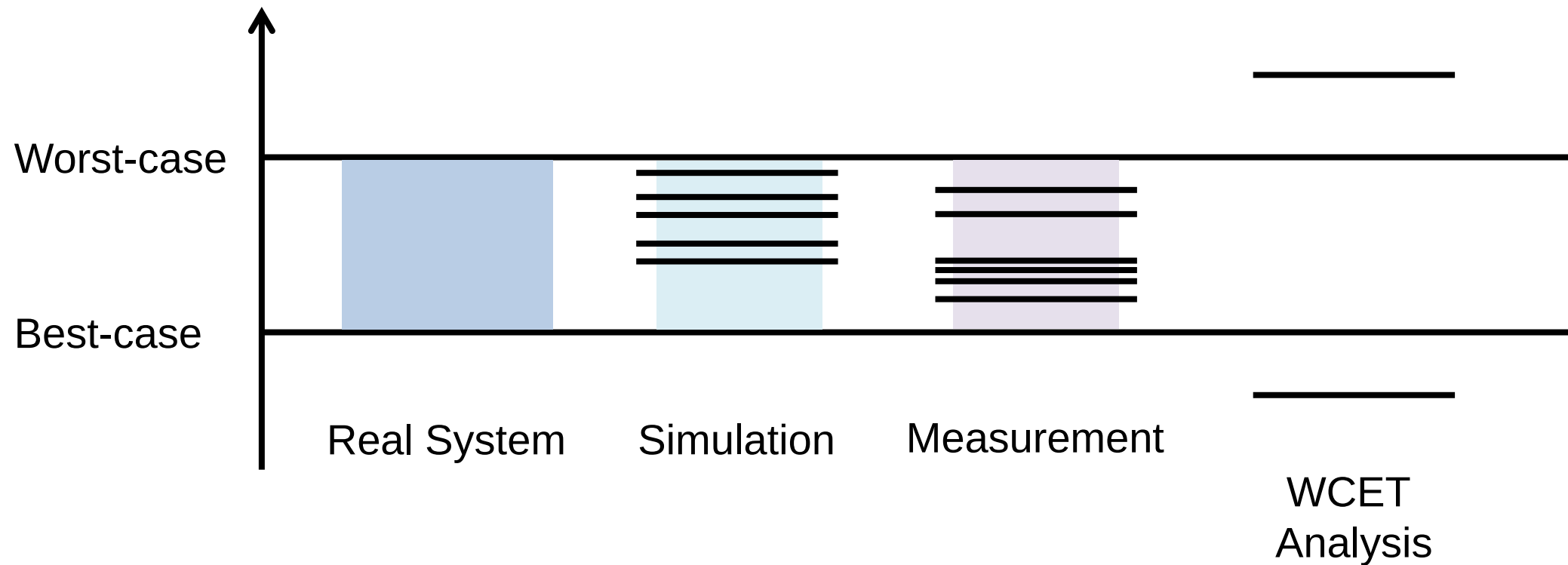


Summary – Overview Design Space Exploration



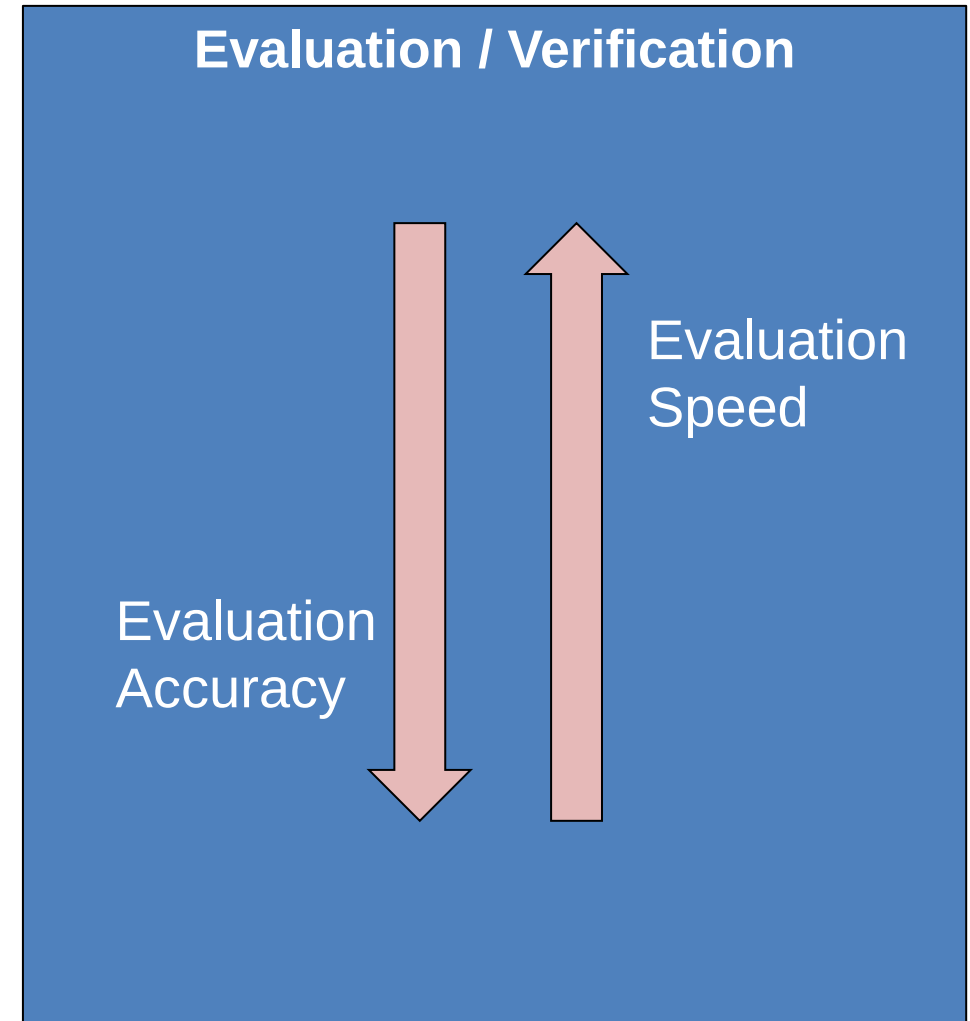
Recap: Tasks – Execution Time

- How to determine the execution time of a task?



Design Evaluation

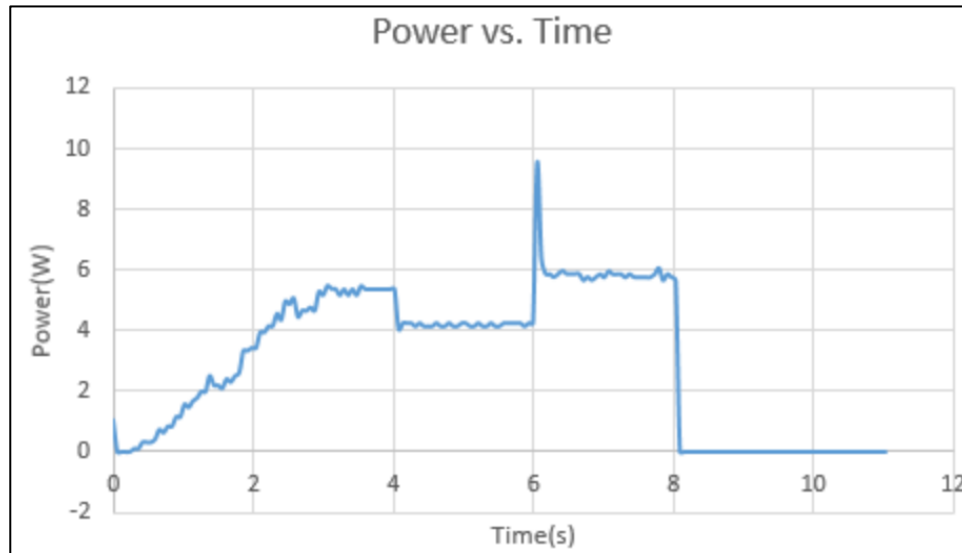
- **Analysis**
 - LoC, Code Size
 - Analytical Processor Models
 - Source Code Execution Time estimators
- **Simulation**
 - System Simulation
 - Processor Simulation
 - Cycle-True Simulation
 - Waveform Simulation
- **Measurements**
 - Really an option (?)



Design Evaluation – Use Models

Analytical models very important for quick evaluation of a specific design

- Convert selected energy measurements into energy models



- Convert selected firmware binaries into models for memory consumption

Memory Map of the image

Image Entry point : 0x0800865

Load Region LR_IROM1 (Base: 0x08000000, Size: 0x00003590, Max: 0x00200000, ABSOLUTE)

Execution Region ER_IROM1 (Exec base: 0x08000000, Load base: 0x08000000, Size: 0x00003588, Max: 0x00200000, ABSOLUTE)

Exec Addr	Load Addr	Size	Type	Attr	Idx	E Section Name	Object
0x08000000	0x08000000	0x000001ac	Data	RO	50	RESET	startup_ctboard.o
0x080001ac	0x080001ac	0x0000003c	Code	RO	119	!!!scatter	c_pu.1(!!scatter.o)
0x080001e8	0x080001e8	0x0000001a	Code	RO	284	!!handler_copy	c_pu.1(!!scatter_copy.o)
0x08000202	0x08000202	0x00000002	PAD				
0x08000204	0x08000204	0x0000001c	Code	RO	286	!!handler_zi	c_pu.1(!!scatter_zi.o)
0x08000220	0x08000220	0x00000002	Code	RO	170	.ARM.Collect\$libinit\$00000000	c_pu.1(libinit.o)
0x08000222	0x08000222	0x00000000	Code	RO	180	.ARM.Collect\$libinit\$00000002	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	182	.ARM.Collect\$libinit\$00000004	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	184	.ARM.Collect\$libinit\$00000006	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	187	.ARM.Collect\$libinit\$0000000C	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	189	.ARM.Collect\$libinit\$0000000E	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	191	.ARM.Collect\$libinit\$00000010	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	194	.ARM.Collect\$libinit\$00000013	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	196	.ARM.Collect\$libinit\$00000015	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	198	.ARM.Collect\$libinit\$00000017	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	200	.ARM.Collect\$libinit\$00000019	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	202	.ARM.Collect\$libinit\$0000001B	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	204	.ARM.Collect\$libinit\$0000001D	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	206	.ARM.Collect\$libinit\$0000001F	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	208	.ARM.Collect\$libinit\$00000021	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	210	.ARM.Collect\$libinit\$00000023	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	212	.ARM.Collect\$libinit\$00000025	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	214	.ARM.Collect\$libinit\$00000027	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	218	.ARM.Collect\$libinit\$0000002E	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	220	.ARM.Collect\$libinit\$00000030	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	222	.ARM.Collect\$libinit\$00000032	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000000	Code	RO	224	.ARM.Collect\$libinit\$00000034	c_pu.1(libinit2.o)
0x08000222	0x08000222	0x00000002	Code	RO	225	.ARM.Collect\$libinit\$00000035	c_pu.1(libinit2.o)
0x08000224	0x08000224	0x00000002	Code	RO	259	.ARM.Collect\$libshutdown\$00000000	c_pu.1(libshutdown.o)
0x08000226	0x08000226	0x00000002	Code	RO	267	.ARM.Collect\$libshutdown\$00000002	c_pu.1(libshutdown.o)

Recap: Design Evaluation – Energy

- To have early estimations of energy consumption, energy models are needed
- First approach based on data sheet (Example CO2 sensor):

4.1.5 The current rating and power consumption

All parameters specified in the following sections refer to the operating conditions unless otherwise specified:
VDD3.3 = 3.3V, VDD12 = 12V, $T_{amb} = 25^{\circ}\text{C}$, % r.H. = 30 %, $p = 1013 \text{ hPa}$.

Table 7 Current rating

Parameter	Symbol	Pin	Values			Unit	Note or Test Condition
			Min.	Typ.	Max.		
Peak current ¹⁾	$I_{peak\ 12}$	VDD12		130	150	mA	
Peak current ¹⁾	$I_{peak\ 3.3}$	VDD3.3		10		mA	
Average current ¹⁾	$I_{avg\ 12}$	VDD12		0.8		mA	
Average current ¹⁾	$I_{avg\ 3.3}$	VDD3.3		6.1		mA	
Average power ¹⁾	P_{avg}						

Power consumption can be optimized further by the product [web page](#).

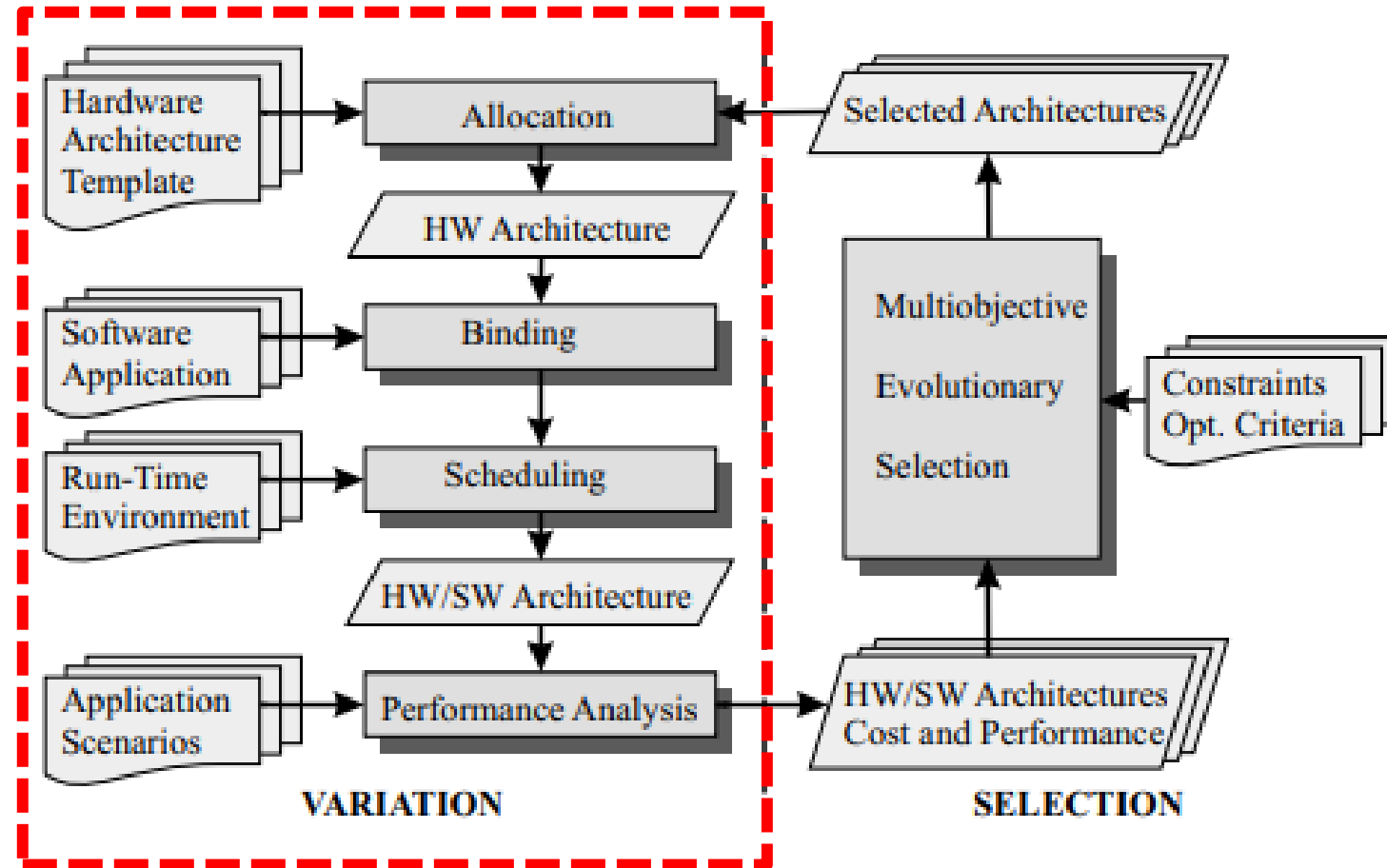
Table 2 Power consumption split

Parameter	Power up	During Measurement		Idle		Unit
	3.3 V	3.3 V	12 V	3.3 V	12 V	
Average current	9	8	70	6	0.5	mA
Average power	29.7	26.4	840	19.8	6	mW
Duration	0.8	0.7	0.2	-		s



210.24 J
per meas.

Summary & Outlook



Summary

- **Design Space Exploration as a multiobjective optimization problem helps to find the implementations that produce the «best» values for all objectives**

- **Stages needed for each possible solution include:**
 - Allocation
 - Binding
 - Runtime Environment Selection (e.g. Scheduling)
 - Estimation by Analytical Methods, Simulation, etc.

- **Success of DSE endeavors heavily depends on the availability of accurate (enough) models for quick objective evaluation**